

University of Economics, Prague  
Faculty of Informatics and Statistics



# **Possibilities of Amazon AWS and Google GCP cloud services in specific usecases**

## **BACHELOR THESIS**

Study programme: Applied Informatics

Field of study: Applied Informatics

Author: Ondřej Michálek

Supervisor: Ing. Tomáš Bruckner, Ph.D.

Prague, May 2020



## Declaration

I hereby declare that I am the sole author of the thesis entitled "*Possibilities of Amazon AWS and Google GCP cloud services in specific usecases*". I duly marked out all quotations. The used literature and sources are stated in the attached list of references.

In Prague on .....

.....

Student signature



## **Acknowledgement**

I hereby wish to express my appreciation and gratitude to the supervisor of my thesis, Ing. Tomáš Bruckner, Ph.D., for the continuous guidance, patience and support during the process of writing this thesis.



## Abstrakt

Poskytovatelé IaaS služeb kteří cílí na enterprise zákazníky a zákazníky středního trhu standardně nabízejí služby vysoké kvality s excelentní dostupností, výkonem, zabezpečením a podporou. [1]. Tato skupina poskytovatelů zahrnuje též Amazon Web Services kteří odstartovali trh cloudových IaaS služeb již v roce 2006, a současně zahrnuje i Google Cloud Platform kteří vstoupili na trh až v roce 2012.

Práce cílí na analýzu a porovnání tří služeb v portfoliu těchto poskytovatelů:

1. Virtuální stroje a jejich využití pro škálování
  - AWS Elastic Compute Cloud
  - GCP Compute Cloud
2. Úložiště v rámci sdílení souborů
  - AWS Simple Storage Service
  - GCP Storage
3. Management přístupů pro zabezpečení dostupnosti služeb a dat
  - AWS IAM
  - GCP IAM

Tyto tři kapitoly obsahují podrobné představení služby, koncept který zastřešuje, a podrobné srovnání kroků potřebných k nastavení této služby. To je nutné pro analýzu rozdílů a podobností poskytovaných služeb a slouží dvěma účelům:

- Srovnání služeb poskytovaných těmito poskytovateli
- Vyhodnocení toho, proč je AWS populárnější než GCP a zdali by zákazníci měli využívat služeb jen jednoho konkrétního poskytovatele

## Klíčová slova

Amazon AWS, Google Cloud Platform, S3, Simple Storage, EC2, Compute Engine, škálování, IAM, řízení přístupu

## Abstract

Cloud IaaS providers that target enterprise and midmarket customers generally offer a high-quality service, with excellent availability, good performance, high security and good customer support [1]. This group of providers contain Amazon Web Services who pioneered the cloud IaaS market in 2006 and also Google Cloud Platform who entered the cloud IaaS market in 2012.

Work aims to analyze and differentiate three services which these providers offer to customers:

1. Virtual machines and their usage in server scaling
  - AWS Elastic Compute Cloud
  - GCP Compute Cloud
2. Storage used while file sharing
  - AWS Simple Storage Service
  - GCP Storage
3. Access Management of services and data
  - AWS IAM
  - GCP IAM

All of these three chapters contain detailed introduction of the service, the concept it roofs and detailed comparison of steps needed to set-up such services. This is needed for analysing the differences and similarities of provided services and serves two purposes:

- Comparison of services provided by these provides
- Evaluation of why is AWS more popular than GCP and if customers should choose either of them

## Keywords

Amazon AWS, Google Cloud Platform, S3, Simple Storage, EC2, Compute Engine, scaling, IAM, access management



# Contents

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>Introduction</b>                                      | <b>17</b> |
| 0.1 Structure . . . . .                                  | 17        |
| 0.2 Goal . . . . .                                       | 18        |
| 0.3 Cloud services . . . . .                             | 18        |
| <b>1 Server scaling</b>                                  | <b>21</b> |
| 1.1 Methodology . . . . .                                | 23        |
| 1.2 AWS . . . . .                                        | 23        |
| 1.2.1 Regions and zones . . . . .                        | 24        |
| 1.2.2 EC2 described . . . . .                            | 25        |
| 1.2.3 Reason for scaling . . . . .                       | 29        |
| 1.3 GCP . . . . .                                        | 40        |
| 1.3.1 Regions and zones . . . . .                        | 40        |
| 1.3.2 GCE described . . . . .                            | 41        |
| 1.3.3 Scaling . . . . .                                  | 44        |
| 1.4 Conclusion . . . . .                                 | 47        |
| <b>2 Simple file sharing across platforms</b>            | <b>49</b> |
| 2.1 Methodology . . . . .                                | 49        |
| 2.1.1 Common approach to share files . . . . .           | 49        |
| 2.1.2 Cloud services approach to share files . . . . .   | 52        |
| 2.2 Comparison of GCP Cloud Storage and AWS S3 . . . . . | 53        |
| 2.2.1 Process of bucket creation . . . . .               | 54        |
| 2.2.2 How can users use this? . . . . .                  | 56        |
| 2.2.3 Pricing . . . . .                                  | 59        |
| 2.3 Conclusion . . . . .                                 | 60        |
| <b>3 Identity and Access Management</b>                  | <b>63</b> |
| 3.1 Methodology . . . . .                                | 63        |
| 3.2 AWS . . . . .                                        | 63        |
| 3.2.1 Access management . . . . .                        | 65        |
| 3.3 GCP . . . . .                                        | 67        |
| 3.3.1 Members . . . . .                                  | 67        |
| 3.3.2 Roles . . . . .                                    | 68        |
| 3.4 Conclusion . . . . .                                 | 69        |
| <b>Conclusion</b>                                        | <b>71</b> |
| <b>List of references</b>                                | <b>73</b> |
| <b>A Navigation of instance management in AWS</b>        | <b>79</b> |

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| <b>B</b> | <b>Navigation of VM management in GCP</b>            | <b>87</b> |
| <b>C</b> | <b>Faulty access to Roles of GCP Service Account</b> | <b>91</b> |

# List of Figures

|      |                                                                                                   |    |
|------|---------------------------------------------------------------------------------------------------|----|
| 1    | On-Premise, IaaS, PaaS, SaaS models comparison (source: bmc.com, June 2019)                       | 19 |
| 1.1  | AWS regions (screenshot from 30.3.2020, source: author)                                           | 24 |
| 1.2  | AWS region selection in console (screenshot from 30.3.2020, source: author)                       | 25 |
| 1.3  | EC2 configure security groups (screenshot from 6.4.2020, source: author)                          | 29 |
| 1.4  | EC2 instance launched (screenshot from 6.4.2020, source: author)                                  | 29 |
| 1.5  | DNS record set in Route 53 (screenshot from 15.4.2020, source: author)                            | 30 |
| 1.6  | AMI creation modal window (screenshot from 18.4.2020, source: author)                             | 31 |
| 1.7  | new instance in EC2 (screenshot from 18.4.2020, source: author)                                   | 31 |
| 1.8  | Load balancer (screenshot from 29.4.2020, source: author)                                         | 32 |
| 1.9  | ELB security groups (screenshot from 29.4.2020, source: author)                                   | 33 |
| 1.10 | ELB target registration (screenshot from 18.4.2020, source: author)                               | 34 |
| 1.11 | ELB created, its DNS name acquired (screenshot from 18.4.2020, source: author)                    | 34 |
| 1.12 | Auto Scaling Group - new launch configuration (screenshot from 29.4.2020, source: author)         | 35 |
| 1.13 | Auto Scaling Group - configure detail (screenshot from 29.4.2020, source: author)                 | 35 |
| 1.14 | Auto Scaling Group - CPU Utilization scaling (screenshot from 29.4.2020, source: author)          | 36 |
| 1.15 | ELB testing (screenshot from 29.4.2020, source: author)                                           | 37 |
| 1.16 | ASG stress testing (screenshot from 29.4.2020, source: author)                                    | 37 |
| 1.17 | CloudWatch alarmed (screenshot from 29.4.2020, source: author)                                    | 38 |
| 1.18 | Target group instance adding (screenshot from 29.4.2020, source: author)                          | 38 |
| 1.19 | Target group healthy state (screenshot from 29.4.2020, source: author)                            | 38 |
| 1.20 | GCP regions (source: cloud.google.com, April 2020)                                                | 41 |
| 1.21 | GCP network (source: cloud.google.com, April 2020)                                                | 41 |
| 1.22 | GCE machine creation navigation (screenshot from 30.4.2020, source: author)                       | 43 |
| 1.23 | GCE create machine (screenshot from 30.4.2020, source: author)                                    | 43 |
| 1.24 | SSH to GCE VM (screenshot from 30.4.2020, source: author)                                         | 44 |
| 1.25 | GCP load balancer test (screenshot from 30.4.2020, source: author)                                | 46 |
| 1.26 | GCP load balancer detail (screenshot from 30.4.2020, source: author)                              | 46 |
| 1.27 | vertical vs horizontal scaling (source: turbonomic.com, June 2017)                                | 46 |
| 2.1  | Gmail file too large notice (screenshot from 23.3.2020, source: author)                           | 51 |
| 2.2  | Google Drive ssh key upload fail (screenshot from 23.3.2020, source: author)                      | 52 |
| 2.3  | GCP bucket creation pricing calculation (screenshot from 23.3.2020, source: author)               | 60 |
| 3.1  | AWS login form (screenshot from 26.4.2020, source: author)                                        | 64 |
| 3.2  | AWS IAM policy for S3 - FullAccess and ReadOnlyAccess (screenshot from 26.4.2020, source: author) | 66 |

|      |                                                                                                                           |    |
|------|---------------------------------------------------------------------------------------------------------------------------|----|
| 3.3  | AWS s3_put policy - list of services (screenshot from 26.4.2020, source: author)                                          | 66 |
| 3.4  | AWS s3_put policy - list of actions (screenshot from 26.4.2020, source: author)                                           | 66 |
| 3.5  | GCP IAM (screenshot from 26.4.2020, source: author)                                                                       | 67 |
| 3.6  | GCP role creation (screenshot from 26.4.2020, source: author)                                                             | 68 |
| 3.7  | GCP quotas (screenshot from 26.4.2020, source: author)                                                                    | 69 |
|      |                                                                                                                           |    |
| A.1  | EC2 where to find (screenshot from 30.3.2020, source: author)                                                             | 79 |
| A.2  | EC2 dashboard (screenshot from 30.3.2020, source: author)                                                                 | 79 |
| A.3  | EC2 launch instance (screenshot from 30.3.2020, source: author)                                                           | 79 |
| A.4  | EC2 select AMI (screenshot from 30.3.2020, source: author)                                                                | 80 |
| A.5  | EC2 select instance type (screenshot from 30.3.2020, source: author)                                                      | 80 |
| A.6  | EC2 configure instance (screenshot from 30.3.2020, source: author)                                                        | 80 |
| A.7  | EC2 add storage (screenshot from 30.3.2020, source: author)                                                               | 80 |
| A.8  | EC2 add tags (screenshot from 30.3.2020, source: author)                                                                  | 81 |
| A.9  | EC2 review (screenshot from 30.3.2020, source: author)                                                                    | 81 |
| A.10 | EC2 SSH key selection (screenshot from 30.3.2020, source: author)                                                         | 81 |
| A.11 | locating the EBS root device (screenshot from 30.3.2020, source: author)                                                  | 82 |
| A.12 | EBS create snapshot (screenshot from 30.3.2020, source: author)                                                           | 82 |
| A.13 | EBS snapshot creation (screenshot from 30.3.2020, source: author)                                                         | 82 |
| A.14 | Snapshot image creation (screenshot from 30.3.2020, source: author)                                                       | 83 |
| A.15 | AMI launch (screenshot from 30.3.2020, source: author)                                                                    | 83 |
| A.16 | EC2 instance IP in route53 (screenshot from 30.3.2020, source: author)                                                    | 83 |
| A.17 | ELB type selection (screenshot from 30.3.2020, source: author)                                                            | 83 |
| A.18 | ELB configuration (screenshot from 30.3.2020, source: author)                                                             | 84 |
| A.19 | ELB HTTPS certificate security settings (screenshot from 30.3.2020, source: author)                                       | 84 |
| A.20 | ELB routing (screenshot from 30.3.2020, source: author)                                                                   | 84 |
| A.21 | Auto Scaling Group - review (screenshot from 30.3.2020, source: author)                                                   | 85 |
| A.22 | Auto Scaling Group - create new launch configuration (screenshot from 30.3.2020, source: author)                          | 85 |
|      |                                                                                                                           |    |
| B.1  | GCP new instance group creation (screenshot from 6.4.2020, source: author)                                                | 87 |
| B.2  | GCP load balancer creation (screenshot from 6.4.2020, source: author)                                                     | 88 |
| B.3  | GCP load balancer backend creation (screenshot from 6.4.2020, source: author)                                             | 88 |
| B.4  | GCP load balancer frontend creation (screenshot from 6.4.2020, source: author)                                            | 89 |
| B.5  | GCP load balancer summarization (screenshot from 30.4.2020, source: author)<br>(screenshot from 6.4.2020, source: author) | 89 |
|      |                                                                                                                           |    |
| C.1  | GCP missing roles in Service Account (screenshot from 26.4.2020, source: author)                                          | 91 |

# List of Tables

- 1    IaaS and Paas example services differences (summarized by the author) . . . . . 18
- 1.1   AWS instance types (data gathered from aws.amazon.com, April 2020, summarized by the author) . . . . . 21
- 1.2   GCP machine types (data gathered from aws.amazon.com, April 2020, summarized by the author) . . . . . 22
- 1.3   T3 instances comparison (data gathered from aws.amazon.com, May 2020, summarized by the author) . . . . . 39
- 2.1   Messenger picture compression in wide aspect ratio (summarized by the author) 50
- 2.2   Messenger picture compression in 16x9 ratio (summarized by the author) . . . . 50



# List of abbreviations

|                                                                |                                            |
|----------------------------------------------------------------|--------------------------------------------|
| <b>AWS</b> Amazon Web Services                                 | <b>RHEL</b> Red Hat Enterprise Linux       |
| <b>GCP</b> Google Cloud Platform                               | <b>EBS</b> Elastic Block Storage           |
| <b>IaaS</b> Infrastructure as a Service                        | <b>CIDR</b> Classless Inter-Domain Routing |
| <b>PaaS</b> Platform as a Service                              | <b>CLI</b> Command Line Interface          |
| <b>EC2</b> Elastic Compute Cloud                               | <b>PV</b> Paravirtual                      |
| <b>GCE</b> Google Cloud Engine                                 | <b>HVM</b> Hardware Virtual Machine        |
| <b>IOPS</b> Input/Output Operations Per Second                 | <b>TTL</b> Time To Live                    |
| <b>VM (VMs)</b> Virtual Machine (Virtual Machines)             | <b>ELB</b> Elastic Load Balancer           |
| <b>AMI (AMIs)</b> Amazon Machine Image (Amazon Machine Images) | <b>GNU</b> GNU's Not Unix!                 |
|                                                                | <b>ACL</b> Access Control List             |
|                                                                | <b>IAM</b> Identity Access Management      |





# Introduction

Since releases of cloud services in the 2000s, the whole market has changed. As time goes by many companies large and small or even individuals, seek the benefits of cloud computing instead of owning their hardware, employing their administrators and spending their own time, money and effort to take care of the availability.

All this outsourcing originated when Amazon created subsidiary Amazon Web Services and introduced Elastic Compute Cloud in August 2006. Cloud computing has been rising ever since. However, Amazon was not the only player in the field; many others followed:

- in April 2008 Google introduced Google App Engine
- in February 2010 Microsoft introduced Microsoft Azure
- in March 2011 IBM introduced IBM SmartCloud
- in June 2012 Oracle introduced Oracle Cloud

Those first on the scene couldn't stay long with what they got - an improvement was inevitable, and so was the hunger in customer demand.

This thesis aims to apprise the reader with Amazon Web Services (AWS) and Google Cloud Platform (GCP) in various situations. Cloud services are a powerful tool but might be intricate to know the ropes. Each provider offers a variety of services reaching from the most commonly used compute engines to highly specialized services for big data, artificial intelligence and machine learning.

## 0.1 Structure

Three topics are going in-depth regarding services of AWS and GCP.

Firstly more technically approached use-cases of specific services provided by AWS and GCP:

- simple file sharing between people and how can user benefit while using cloud services
- effortless scaling of architecture ensuring almost permanent stability

Then a common and a vital topic for both service providers and the users:

- identity and access management - what are options for limiting users for ensuring a secure environment

Each of these chapters is introduced on its own, containing its methodology of reaching a set goal and its conclusion for achieving contained units, logically divided by topic.

## 0.2 Goal

The goal of this thesis is to analyze cloud services that are available in the portfolio of two giants on the market - Google and Amazon. How both of them target their services and why one is more profitable than the other.

In today's world technology develops faster and faster, and it's often challenging to keep up with it. In the following chapters are described and analyzed a few concepts and possibilities, written in such a spirit that the comparisons does not limit a person with limited knowledge of AWS/GCP, and also teaches the basics of described technologies.

Also, question about security needs to be addressed. Every service must have some security mechanisms in place - what are the key approaches how user access can be assigned. These objectives are essential to be familiar with while using any services of any provider; AWS/GCP is no exception here.

## 0.3 Cloud services

Cloud services and especially cloud computing is used in many fields for many use-cases. Some of these services are offered as a "Platform as a Service" where the only user objective is to manage a small part of a more complex functioning system. Yet, others are an "Infrastructure as a Service", allowing user deeper variability.

There is a difference of this for customer objectives while running an application:

Table 1: IaaS and Paas example services differences (summarized by the author)

| Approach | AWS service       | GCP service    | Cusomer objective                                                 |
|----------|-------------------|----------------|-------------------------------------------------------------------|
| PaaS     | Elastic Beanstalk | App Engine     | code<br>data storage                                              |
| IaaS     | EC2               | Compute Engine | code<br>data storage<br>runtime<br>middleware<br>operating system |

A more detailed structure can be seen in the picture bellow

Sometimes the customer wants a simple solution that makes his application reachable from the internet and a solution that scales itself and does not care about lower-level technologies involved. On the other hand, sometimes a customer lacks further optimization or needs very specific requirements which can be achieved only on PaaS solution.

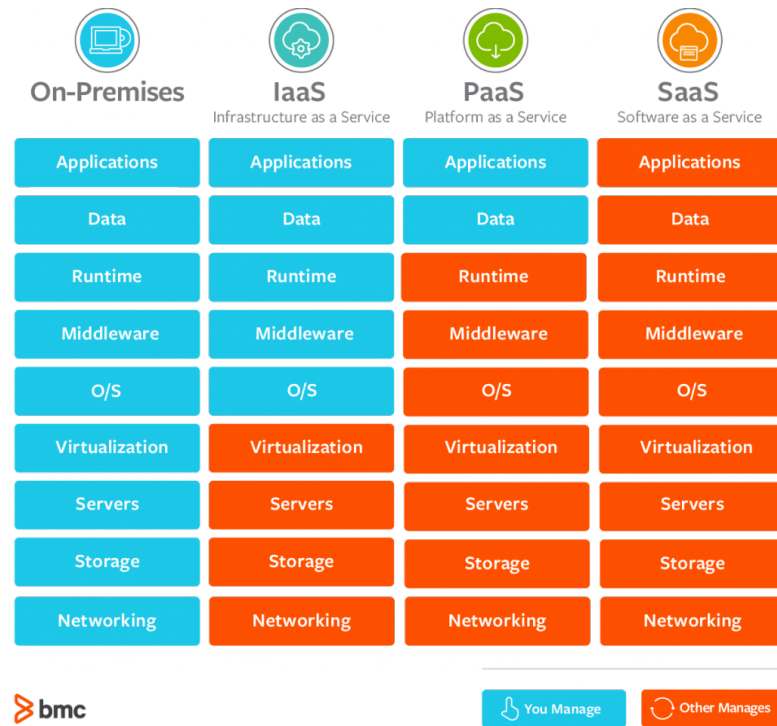


Figure 1: On-Premise, IaaS, PaaS, SaaS models comparison (source: bmc.com, June 2019)

In today world, businesses depend on the provision of computational power, and more than ever, there is a need to transform data and to serve content to users.

Many of these businesses have a web page as a frontpage of what they do, of services they provide and the products they sell. This internet gateways might be a static HTML website or an intricate application with an extensive backend.

There are 1.2 billion websites (reported by Netcraft [2]) as of 2019, and each one of them must be hosted somewhere, on physical storage and be provided with hardware and software and resources.

In the past and the present, these resources are provided either by the user himself or by an external provider. These providers help users to handle all the technical aspects so the user can focus his efforts away from complicated and intricate technical knowledge.

As time went by, providers became more popular, and users naturally gravitated towards bigger companies, as they became more and more known in society. They started to offer more and more services to attract a broader spectrum of customers. "The market views both AWS and Microsoft as being general-purpose providers capable of supporting a broad range of workloads. Google is making steady progress in terms of enterprise adoption" [1]

The thesis aims at AWS as the top leader in cloud services provider, and Google as a massive company with immense impact on the internet, which is also gaining a foothold in the market.



# 1. Server scaling

Most companies run their servers in the cloud in such a way that they spin up a server, they set the environment accordingly to their needs and keep it running until a breakdown happens or until there is a need to change. User can always choose how fast the server will be accordingly to the resources it uses. After all, the server is only a virtual machine running on actual hardware. These resources are shared over multiple virtual machines allocated with a predefined amount of CPU cores and RAM size. Some providers allow users to choose GPU capabilities as well as storage.

Amazon calls these mostly as "instances" because it is a running instance of an instance type, whereas Google calls these "VMs" as virtual machines, probably for lowering the complexity. After all, it is just a virtual machine running on top of some hardware. Similarly, the types are named in a different jargon as well. AWS terminology is "instance types", and Google terminology is "machine types". It seems that both providers match on some naming, but still keep some amount of distinction.

For example, Amazon allows user to choose from multiple types of virtual machines in their Elastic Compute Cloud, mostly known as EC2. these are called families, and each of them has different kinds of types based on AWS documentation [3]

Table 1.1: AWS instance types (data gathered from [aws.amazon.com](https://aws.amazon.com/ec2/instance-types/), April 2020, summarized by the author)

| Instance type         | Instance type name                          |
|-----------------------|---------------------------------------------|
| General Purpose       | a1, t3, t3a, t2, m6g, m5, m5a, m5n, m4      |
| Compute Optimized     | c5, c5n, c4                                 |
| Memory Optimized      | r5, r5a, r5n, r4, x1e, x1, high memory, z1d |
| Accelerated Computing | p3, p2, inf1, g4, g3, f1                    |
| Storage Optimized     | i3, i3en, d2, h1                            |

Here you can see that Amazon offers a lot to choose from, and every user will be able to choose what suits him best. This variety of options is indeed extraordinary, and then the user must have a certain amount of knowledge to know what might work best for his use case. Or plenty of time to try the trial-and-error approach

Instance types are not static though, they change throughout the time as hardware and customer needs evolve, and because of this, so do the instance types. If there is a new generation of hardware introduced and used to upgrade a specific instance type, usually the instance type version is upgraded, and the old hardware still stays and is still used and provided to customers until possible. This generation diversification is the reason why there are instances such as r5 and r4 (and now discontinued r3) which mainly differ in their processor

architecture.

- r3 - Intel Xeon E5-2670 v2 (Ivy Bridge)
- r4 - Intel Xeon E5-2686 v4 (Broadwell)
- r5 - Intel Xeon Platinum 8175 (Skylake)

If there is an "a" suffix in the instance type name, it means the instance is running on AMD CPUs. Another thing to remember is that AWS instance types differ in network performance as a different network interface is allocated upon starting the instance based on its type. All this information provides AWS in their documentation, or even better comparison is available on [ec2instances.info](https://ec2instances.info) [4], which is a simple front-end application containing all information needed, including cost.

Second example are the offers Google provides in their configurator [5]:

- General purpose - n2, s2d, e2
- Compute optimized - c2
- Memory optimized - m2

Table 1.2: GCP machine types (data gathered from [aws.amazon.com](https://aws.amazon.com), April 2020, summarized by the author)

| Machine type      | Machine family name |
|-------------------|---------------------|
| General purpose   | n2, s2d, e2         |
| Compute optimized | c2                  |
| Memory optimized  | m2                  |

After choosing family type there is always the option to configure machine size, meaning how many cores and how much memory the instance will be provided with. These are usually in steps of twice the amount as previous configuration. As an example lets show the GCP N2 standard machine types [6]:

- n2-standard-2: 2 vCPU, 8GB RAM
- n2-standard-4: 4 vCPU, 16GB RAM
- n2-standard-8: 8 vCPU, 32GB RAM
- n2-standard-16: 16 vCPU, 64GB RAM
- n2-standard-32: 32 vCPU, 128GB RAM
- n2-standard-48: 48 vCPU, 192GB RAM
- n2-standard-64: 64 vCPU, 256GB RAM
- n2-standard-80: 80 vCPU, 320GB RAM

Apart from standard, High-memory and High-CPU machine types, GCP allows user to chose specific configuration with two simple sliders, one for CPU cores, the other for memory.

After all this configuration, research and decision-making of CPU/RAM, there is the option

to attach network storage. These vary in sizes, speeds and additional capabilities such as IOPS (input/output operations per second), connection to multiple VMs at once, etc.

After all, running an instance is not easy. There is no button saying "Start an instance". Not only because of the immense variety of options to choose from.

## 1.1 Methodology

In the following sub-chapters is included a more detailed description of features and capabilities AWS and GCP provide for users. AWS approach is introduced first while describing some of the important details, topics and concepts, and then GCP is approached similarly.

The core of the method is introduction and analysis in web interfaces of these two providers, most importantly, the ease of server obtainability. Some examples are also given in the CLI interface. Based on this introduction, a comparison is set, and a choice of one over the other is evaluated.

Important topics are introduced containing the virtual server, which is the core of server scaling, the process of creation of its instance and other steps which are tightly connected with the topic. These steps include data storage, security, access, networking and operating systems and more. The two providers are also introduced including the infrastructure they operate on.

Then the server scaling is being handled. Concepts of vertical and horizontal scaling are introduced, a comparison is explained, and then horizontal scaling is handled deeper. Firstly the setup required, then the important concepts behind the technology and acquaintance with the technology involved. This also contains other approaches of responding to a variable load.

Lastly, the actual testing of the previously set environment is performed to determine whether it can handle increased load and balance the incoming traffic accordingly. The test is conducted based on observation of the server IP addresses returned by the load balancer.

## 1.2 AWS

AWS offers few approaches of scaling of instances where all are suitable for something a bit different. First, let's go a bit into detail of the AWS ecosystem and then to EC2 instances, the configuration process and access to them.

### 1.2.1 Regions and zones

There are multiple data centres all around the world. Each of these locations is separated physically from each other. This separation achieves fault tolerance, stability and possibly also lower latency. AWS Global Infrastructure Map shows there regions in great detail since 2006 when AWS was launched, introducing the Northern Virginia region.



Figure 1.1: AWS regions (screenshot from 30.3.2020, source: author)

If a user owns a business which grew in Europe and suddenly a significant amount of clients are in Australia, the simplest way to achieve lower latency for them is to set up this second environment in Sydney. Clients approach content from servers which are closer to them and in case of a fatal breakdown of one of these data centres, the other can provide sufficient temporary traffic.

These are entirely separated data centres, so all the traffic between them costs money. Imagine one scenario where we're setting up a second environment in Sydney, and it's necessary to duplicate all the data stored in a database. This duplication will cost money, as there is data traffic across half of the globe is necessary. On the other hand, if the data transfer is within a region, the cost is significantly lower.

This separation is visible in the cloud console as well, as the interface needs to reflect differences in the features provided.

There is no rule that every region offers all the features AWS offers as a whole. Usually, the older regions have more features and also more zones than newer regions. For example, Northern Virginia and Oregon regions have more features available than other regions in the USA. The same applies to Ireland compared to Frankfurt, London, Paris and Stockholm in Europe.

I've mentioned zones many times. The full name is "Availability Zones", and they are a way to distribute instances. "If you distribute your instances across multiple Availability Zones and one instance fails, you can design your application so that an instance in another Availability Zone can handle requests." [7]



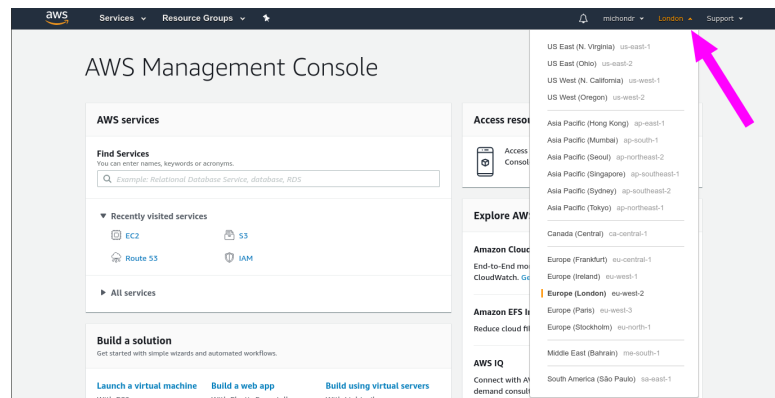


Figure 1.2: AWS region selection in console (screenshot from 30.3.2020, source: author)

## 1.2.2 EC2 described

In the introduction of this chapter, short info about EC2 instances has been given. Now is the time to go more in detail of what it means to create a fresh instance, or to spin it up, as often referred.

### EC2 instance types

First of all, let's define 3 types of the instance operation which are highly reflecting its pricing.

- On-Demand: "On-Demand instances let you pay for computing capacity by the hour or second with no long-term commitments. This frees you from the costs and complexities of planning, purchasing, and maintaining hardware and transforms what are commonly large fixed costs into much smaller variable costs." [8]
- Spot instance: "Amazon EC2 Spot Instances let you take advantage of unused EC2 capacity in the AWS cloud. Spot Instances are available at up to a 90% discount compared to On-Demand prices. You can use Spot Instances for various stateless, fault-tolerant, or flexible applications." [9]
- Reserved Instances: "Amazon EC2 Reserved Instances (RI) provide a significant discount (up to 75%) compared to On-Demand pricing and provide a capacity reservation when used in a specific Availability Zone." [10]

Amazon owns a massive amount of computing capacity in its data centres to meet demand. With the way the "On-Demand" instances are structured, anybody can spin up an instance any time, and AWS must handle that demand to keep the instances running. Also, the instance must stay running until the user decides otherwise.

To keep the demand more stable, Amazon provides an option to "rent" the instance for long-term periods in exchange of lowering the price. These are the "Reserved instances".

Last but not least, keeping the servers unused to handle potential demand would be inefficient. There is a way of selling this otherwise unused computing power, and that is the concept

of "Spot Instances". These instances are for a fraction of the hourly cost of the on-demand instances. The only downside is that at any moment, AWS can evaluate these instance to be terminated. This evaluation works in such a way that while creating a spot instance via so-called spot request, the user sets a bid which is the maximum price he is willing to pay. If the demand for this instance goes up, the price goes up accordingly until the demand is met. If the current price overcomes set bid price, the instance is terminated within 2 minutes. There is an endpoint available that serves the necessary information about instance termination so that automatic processes can be triggered to overcome this state. [11]

## Spinning up a EC2 instance

Most often a user will manage anything in AWS through their console. This console is an amazon-developed web interface where users can manage all the AWS services - used and available. Of course, there are more ways to manage these services, such as the `aws-cli` command-line interface.

Upon clicking "Launch instance" in EC2 dashboard, the user is presented with a 7-step form of instance creation. First of these steps is AMI selection. AMI stands for "Amazon Machine Image" and are defining the operating system and its stage of the instance. Depending on needs, one of many predefined AMIs can be selected. There are many options to select from depending solely on your needs. There is always available a stock (base) version of any OS and then derivatives built upon these base versions.

base versions contain probably every OS any user could desire in many versions:

- Amazon Linux - OS build on top of Linux kernel allegedly bases on RHEL distribution (Red Hat Enterprise) [12]
- Cent OS
- Debian
- Fedora
- Gentoo
- openSUSE
- Other Linux
- Red Hat
- SUSE Linux
- Ubuntu
- Windows

Remember that AMI is the state of the operating system. So apart from the base versions many of the AMIs are available in many flavours with pre-installed programs. These versions include Ubuntu deep learning AMIs with a variety of pre-installed python libraries, or Windows AMIs with SQL server.

Most importantly, any user can create their own AMIs as an image of a disk. This approach

can be beneficial in a few ways:

- as a backup - AMI creation can be automatized, so new one is created for example every day. In case of some problems with a running instance, a new one can be spun up with the most recent AMI, and everything is up and running. This kind of problem can sometimes happen, as there might be an issue with the hardware on which the instance is running. By running a new instance, this issue might resolve effortlessly.
- as a server set-up shortcut - if there is a constant need for some features and clean installation is not necessary, an AMI can be the excellent starting point of server configuration, as most of the work has already been done.

After selecting an AMI, selection of instance type follows. This table is the place where most of the research comes in, as the user must decide which type reflects his needs the most. Unfortunately, pricing is not available here, as AWS is more structured towards the approach "I choose what I need and don't care that much about how expensive it is".

Also, one of the points to mention is the column with network performance. Amazon has a fuzzy definition of network performance, as they categorize it in a few groups:

- Low to Moderate
- Moderate
- High
- Up to 5 Gigabit
- Up to 10 Gigabit
- 10 Gigabit
- 12 Gigabit
- 20 Gigabit
- 25 Gigabit
- 50 Gigabit
- 100 Gigabit

There is no obvious guarantee of the transfer speeds, and only final testing will show proper results if said speed is sufficient. There are multiple performance tests available on the internet [13]

Next is a configuration of instance details. the important settings:

- the number of instances - if the user needs to start more than one instance of the type, here is the place to do so.
- request spot instance - the default on-demand instance type can be changed to spot request to lower the cost in exchange to the probability of instance termination
- T2/T3 Unlimited - allowing higher processing power of the instance. This option is covered further on as one of the ways of scaling.

Storage adding can be configured while starting the instance or while the instance is running

(the disk must be mounted, and filesystem created).

Some instance types are already pre-attached with a drive, as the 8GiB drive in the following picture. User can then add many other volumes of the multiple types. One of those is EBS standing for "Elastic Block Storage" which is essentially a network disk attached to the instance which is independent on it. The discs will remain available upon instance termination and thus can be attached to a different instance. This preservation of disks is crucial while changing instance types and optimizing for the required size of the instance.

Multiple EBS discs can be attached to a single EC2 instance but not the other way around. EBS disk can be attached to only a single instance.

There are multiple volume types to choose from, the main difference being the speed and price. While the volume type is essential, interestingly, the bigger the volume is, the faster it is.

"The requested number of I/O operations per second that the volume can support. For Provisioned IOPS (SSD) volumes, you can provision up to 50 IOPS per GiB. For General Purpose (SSD) volumes, baseline performance is 3 IOPS per GiB, with a minimum of 100 IOPS and a maximum of 16000 IOPS" [14]

More in-depth detailed analysis of EBS types and speed is available in the documentation [15]

Adding tags is crucial for situations where multiple instances are present in any form (running, terminated or state-changing), and they become confusing. AWS allows user to tag the EC2 instance and the EBS drive in a single step. Multiple tags can be added at the same time.

The last step before the final review is the security groups. User can create new or select one created previously. Security groups allow limiting access to the machine itself. They work oppositely than restricting, as the user allows access. By default, all access is forbidden apart from SSH on port 22. Every rule has a set of attributes:

- type - set of predefined rules like SSH on port 22, HTTP on 80, HTTPS on 443 or MYSQL on 3306 and many others. apart from these commonly known rules, user can select different TCP/UDP/ICMP protocols or define a custom type.
- protocol - automatically selected or possible to edit in case of a custom rule
- port range - automatically selected or possible to edit as a number or a range in case of a custom rule
- source - option to set a source of request which will be allowed. A source can be set a specific IP address (IPv4 or IPv6), IP range in CIDR notation (e.g. 203.0.113.5/32) or a different security group (prefixed with the account ID and a forward slash, e.g. 111122223333/OtherSecurityGroup)
- description - as usual, a description is fundamental, as it is impossible to remember what each of the rules allow when multiple of them exist.

These rules are, of course, a great approach to security, as it allows each end every "hole" to the instance to be minimized, reasoned, explicitly defined and tightened as much as possible. For example, in production use the access to ports 80 and 443 for HTTP and HTTPS respectively is allowed, and also to all needed technologies like databases. Then developers need their access to the server via SSH, the office IP is allowed on port 22.

**Step 6: Configure Security Group**  
 A security group is a set of firewall rules that control the traffic for your instance. On this page, you can add rules to allow specific traffic to reach your instance. For example, if you want to set up a web server and allow Internet traffic to reach your instance, add rules that allow unrestricted access to the HTTP and HTTPS ports. You can create a new security group or select from an existing one below. [Learn more](#) about Amazon EC2 security groups.

Assign a security group: ☒ Create a new security group ☐ Select an existing security group

Security group name:

Description:

| Type  | Protocol | Port Range | Source                | Description                |
|-------|----------|------------|-----------------------|----------------------------|
| HTTP  | TCP      | 80         | Custom 0.0.0.0/0, :/0 | e.g. SSH for Admin Desktop |
| HTTPS | TCP      | 443        | Custom 0.0.0.0/0, :/0 | e.g. SSH for Admin Desktop |
| SSH   | TCP      | 22         | My IP 82.113.52.33/32 | e.g. SSH for Admin Desktop |

Figure 1.3: EC2 configure security groups (screenshot from 6.4.2020, source: author)

The last step is the review of all the steps. The only thing necessary is to push the "Launch" button. After that, a modal window for an SSH key appears. This prompt is to ensure that admin will be able to SSH to the server via CLI and set-up the environment. AWS enables the user to choose from keys which are already created/uploaded to AWS or to create a new key pair. In that case, a copy of the RSA public key is stored in AWS, and RSA private key is downloaded in a .pem file

After that, the instance is started, and its state can be monitored in the instances list.

| Instance-name     | Instance ID          | Instance Type | Availability Zone | Instance State | Status Checks  | Alarm Status | Public DNS (IPv4)     |
|-------------------|----------------------|---------------|-------------------|----------------|----------------|--------------|-----------------------|
| preprod autoscale | i-0d7a715a3d71a24... | t2.micro      | eu-west-2a        | running        | 2/2 checks ... | None         | ec2-35-176-6-37.eu-we |

Figure 1.4: EC2 instance launched (screenshot from 6.4.2020, source: author)

### 1.2.3 Reason for scaling

Let's define the possible problem - consider a scenario where there is a system developed which awaits request on a server. This system will be built solely on AWS.

An EC2 instance is running, and it is assigned a public IPv4 address, 3.9.171.175 in this example.

A request must be transferred to the instance. For that, a DNS (Domain Name System) record must exist. AWS has a service for DNS management, traffic management, availability monitoring and domain registration called Route 53, which is neatly integrated with other AWS services.

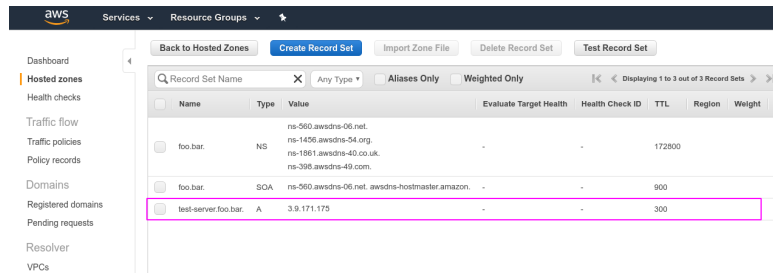


Figure 1.5: DNS record set in Route 53 (screenshot from 15.4.2020, source: author)

Now upon calling `test-server.foo.bar` a request is routed to IP `3.9.171.175` which the instance if the access is allowed in security groups.

If this is an HTTP request, a webserver such as Apache or Nginx handles it. The web servers will use some resources of the machine, such as CPU and RAM for some amount of time. While one request is not a problem, many of them can be. If the available capacity of hardware resources is exhausted, other requests wait in a queue or are refused (depending on configuration). The bigger problem is a variable load, where resource depletion can be estimated only with difficulty.

In cases of such behaviour and load, developers have few options for optimization:

- scale vertically
- scale horizontally
- use tricks AWS offer

These options are the most essential part of this chapter, as it introduces specific ways to issue high-load scenarios.

## Vertical scaling

Vertical scaling is a term describing the scaling of each of the deployed instances towards their size. If 4 CPU cores cannot handle the load, then 8 might do.

As there is a need to scale up, a larger instance must replace the smaller. If done correctly, this can result in no downtime of an application.

First and foremost, new AMI must be created. As previously mentioned, AMI is an image of the disk. This can be done by navigating to the root EBS volume of the instance.

The device will be preselected, so the user clicks on "Actions" and "Create Snapshot". After filling in the description, click on "Create Snapshot" is necessary. In "Snapshots" selection, the created snapshot must be located, selected, and the next step is to select "Create Image" in "Actions".

User is presented with a modal window asking for information about the created AMI. It is

good to fill in a name and a description so that the reason for existence if this AMI is not lost in the future. The vital choice here is the "Virtualization type". Different instance types allow different virtualization types as the info describes:

When an instance is launched from an AMI, it uses either paravirtual (PV) or hardware virtual machine (HVM) virtualization. HVM virtualization uses hardware-assist technology provided by the AWS platform. With HVM virtualization, the guest VM runs as if it were on a native hardware platform, except that it still uses PV network and storage drivers for improved performance. Some instance types support both PV and HVM, while others support only one or the other.

A best practice here is to look up allowed type on `ec2instances.info`.

| Volume Type | Device    | Snapshot               | Size (GiB) | Volume Type        | IOPS       | Throughput (MB/s) | Delete on Termination               | Encrypted     |
|-------------|-----------|------------------------|------------|--------------------|------------|-------------------|-------------------------------------|---------------|
| Root        | /dev/sda1 | snap-094437504c7ca7039 | 8          | General Purpose S3 | 100 / 3000 | N/A               | <input checked="" type="checkbox"/> | Not Encrypted |

Figure 1.6: AMI creation modal window (screenshot from 18.4.2020, source: author)

Last thing for new instance creation is to select "Launch" in AMIs selection, and the user follows redirection to instance creation wizard.

The instance is immediately launched. It is assigned new IP address upon starting. Instance state is pending, and status check is initializing while the instance is not fully operational.

| Instance-name     | Instance ID          | Instance Type | Availability Zone | Instance State | Status Checks  | Alarm Status | Public IP |
|-------------------|----------------------|---------------|-------------------|----------------|----------------|--------------|-----------|
| preprod autoscale | i-057a715a3d71a24... | t2.micro      | eu-west-2a        | running        | 2/2 checks ... | None         | ec2-3     |
|                   | i-0e43a57909f434c42  | t2.medium     | eu-west-2a        | pending        | Initializing   | None         | ec2-3     |

| Description                      | Status Checks           | Monitoring                          | Tags     |
|----------------------------------|-------------------------|-------------------------------------|----------|
| Instance ID: i-0e43a57909f434c42 | Instance state: pending | Opt-in to AWS Compute Optimizer for | Findings |

| Public DNS (IPv4)                                 | IPv4 Public IP | IPv6 IPs | Elastic IPs |
|---------------------------------------------------|----------------|----------|-------------|
| ec2-35-176-91-118.eu-west-2.compute.amazonaws.com | 35.176.91.118  | -        | -           |

Figure 1.7: new instance in EC2 (screenshot from 18.4.2020, source: author)

When the instance is fully operational, the last step is to switch the IP address in Route 53. When TTL ends, new DNS info is loaded and based on it, the requests will be sent to the new instance instead of the old one.

Vertical scaling is an excellent solution in some cases, but it cannot be easily automated. Nonetheless it is sufficient as an ad-hoc one-time solution. With vertical scaling, it is impossible to reflect variable needs and variable traffic. For that, the horizontal scaling is the way to go.

## Horizontal scaling

Horizontal scaling takes a different, more complicated approach to the request handling. Instead of scaling the size of the instance, the number of instances is scaled. This way, more or less instance can be started or terminated when needed.

There is, of course, utterly different set-up needed. In front of the instances a load balancer is needed, which divides the requests to each server.

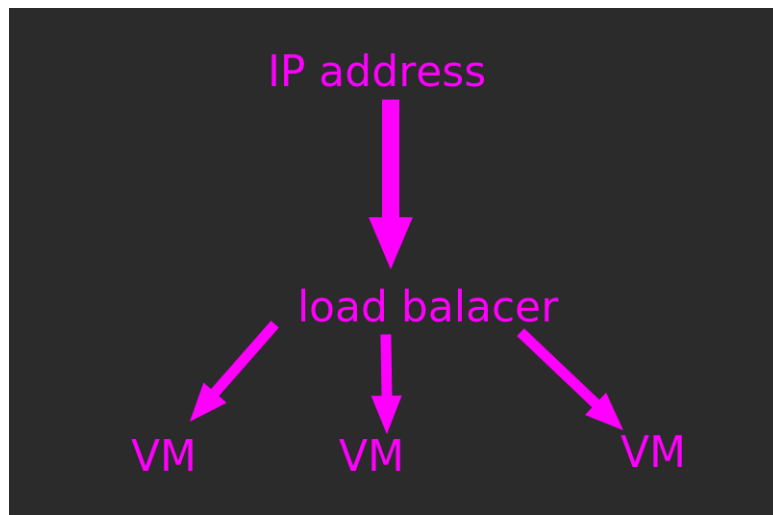


Figure 1.8: Load balancer (screenshot from 29.4.2020, source: author)

To create one, in "Load Balancers" user clicks "Create Load Balancer", then he is presented with a type of ELB (elastic load balancer). For application usage such as HTTP/HTTPS, an Application Load Balancer is an option.

Correct configuration of the ELB is essential. The request comes to IP of load balancer if this is used to handle external requests to some endpoint, the scheme is "internet-facing". User must not forget that HTTP and HTTPS protocols are used, so it is necessary to specify these protocols in Listeners section. Also, mind the Availability Zones section. This input is to choose subnets in which the load balancer routes the traffic.

Next step is available only while setting up the ELB with support for HTTPS as there is a certificate needed. It works in similar ways as setting up a certificate in Linux, for example, with Certbot. As the AWS console is the only way to manage the load balancer, the certificate must be uploaded to the console.

In Security group configuration is possible to create a new one or use an existing group. This



option is mainly to restrict access to the ELB via IP rules.

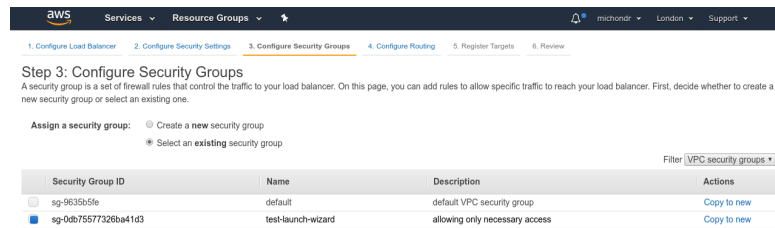


Figure 1.9: ELB security groups (screenshot from 29.4.2020, source: author)

Routing configuration allows to set up a target group. This target group is a set of devices behind the ELB, a set of devices the load is distributed between. Usually, these are EC2 instances, but a Lambda function can be addressed as well.

Let's assume the instance type is chosen. All instances in the target group are seen as healthy at the start. ELB divides load equally between all of them. If any of the instances fail, the unhealthy threshold is reached and the instance it is marked as unhealthy. Usual rules for this distinction is are HTTP return codes. These health checks can be set up quite expressively:

- protocol and path - HTTP or HTTPS on a particular path of URL
- port - the same port as traffic flows through or overridden to a different one
- healthy threshold - The number of consecutive health checks successes required before considering an unhealthy target healthy
- unhealthy threshold - The number of consecutive health check failures required before considering a target unhealthy
- timeout - The amount of time, in seconds, during which no response means a failed health check
- interval - The approximate amount of time between health checks of an individual target
- success codes - The HTTP codes to use when checking for a successful response from a target

In targets registration, the necessary filling up of the target group happens. It is indispensable to click on the instance and then "Add to registered". This way, the instance registers and requests will be forwarded to it.

After review and creation, the load balancer is created and active and is assigned its DNS name. This name is an "A record" possible to be set in Route 53.

At this point the ELB is set-up, so by calling this URL the traffic is transferred to one of the instances.

Now the vital part happens. Having one instance in ELB makes no difference compared to having a single instance. Of course, multiple instances can be added, which solves the issue with variable load. But scaling one instance up will increase costs even when no extra compute power is needed. Horizontal scaling solves this issue beautifully with Auto Scaling

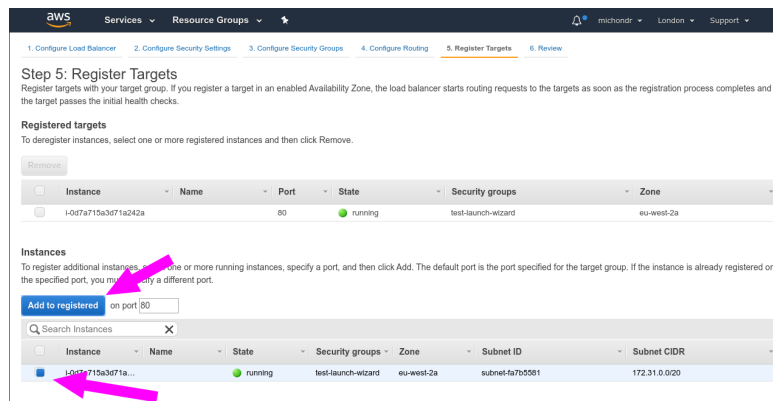


Figure 1.10: ELB target registration (screenshot from 18.4.2020, source: author)

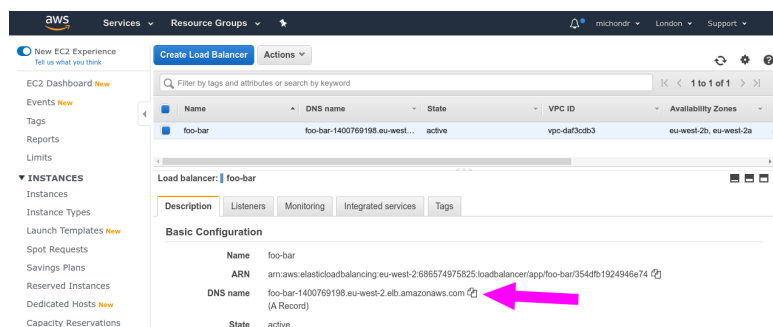


Figure 1.11: ELB created, its DNS name acquired (screenshot from 18.4.2020, source: author)

Groups.

These can scale up or down based on different metrics and reflect the needs of the system almost in real-time. This scaling can work solely - thanks to automation and also fast creation of instances based on Launch Configurations created ahead of time. Amazon released a new version recently called Launch Template. The difference is that templates allow for versioning and combining of instance types - On-Demand and Spot Instances [16]. The same can be achieved while creating multiple Launch Configurations. Those are easier to set-up, as the process is identical to instance creation. This example will follow with the creation of a Launch Configuration.

On the first page of the configuration, it is necessary to select correct values so that the autoscaling group correctly operates with the Load Balancer:

- subnets in which the ELB operates
- check option for receiving traffic from ELB
- selecting previously created Target Group
- selecting Health Check Type to "ELB"

Now the core of autoscaling - that is the scaling automatically based on some value. The simplest is the CPU load which will be used in the example below. Notice the target value of 80, meaning: if metrics "Average CPU Utilization" reaches a value of 80% another instance will be started up to a count of 3. However, they will not be added to the autoscaling group

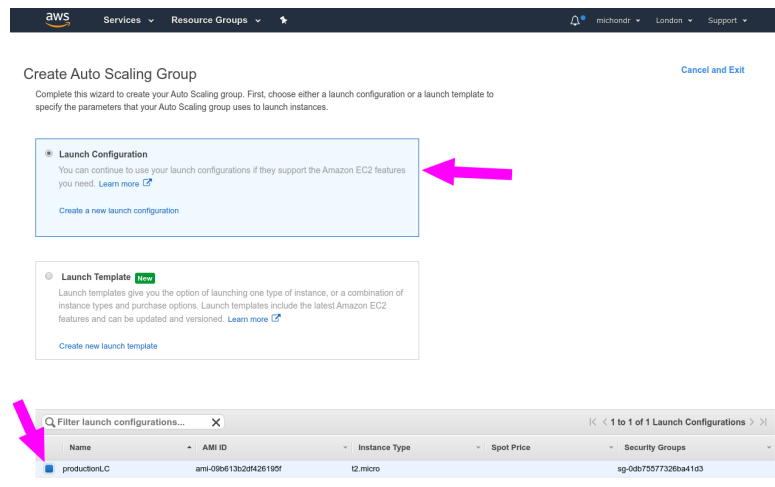


Figure 1.12: Auto Scaling Group - new launch configuration (screenshot from 29.4.2020, source: author)

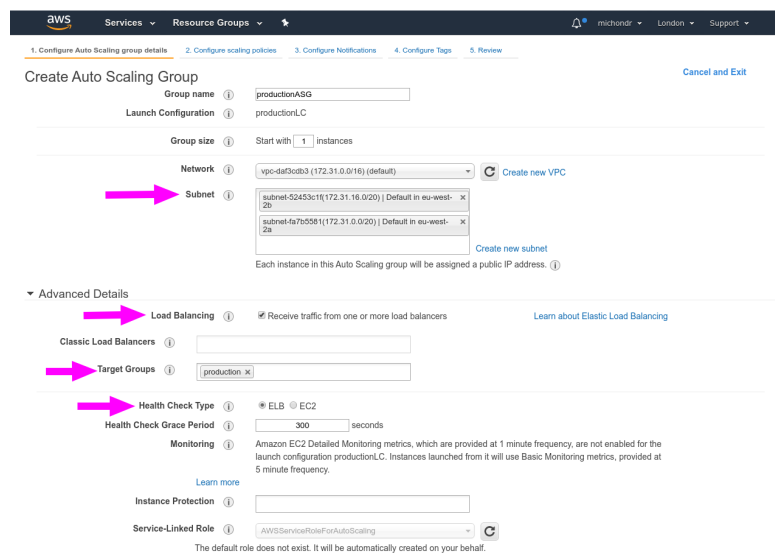


Figure 1.13: Auto Scaling Group - configure detail (screenshot from 29.4.2020, source: author)

immediately as they need some time to become fully operational, 300 seconds in this case.

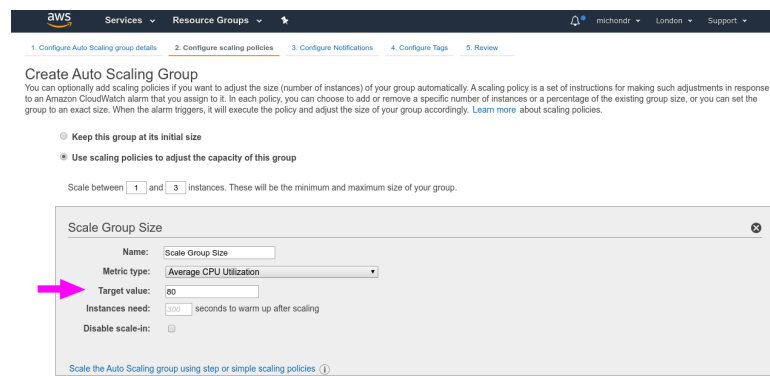


Figure 1.14: Auto Scaling Group - CPU Utilization scaling (screenshot from 29.4.2020, source: author)

As I mentioned previously, this is the simplest way to set-up an autoscaling group. User can define separate rules for increasing and decreasing group size. The use-case is that this behaviour reflects increasing or decreasing demand for more computing power. This demand might be caused by many requests flooding the system, or some backend data computation process is triggered and needs more resources. Auto Scaling Groups do not need to be behind a load balancer but can exist without one, dealing with some kind of a backend task.

Nonetheless, more advanced rules can be defined using the CloudWatch Management Console. CloudWatch is a tool which brings metrics from all services AWS offers to one place. Here alarms can be created based on one or more metrics. Because each metric is a set of data describing some behaviour of a service in time, these metrics can be combined or mathematically modified. After all of this, a new alarm is created in CloudWatch, which is primarily designed to alert a developer or a DevOp; however, further steps can be introduced, such as triggering autoscaling behaviour.

In further steps of autoscaling, wizard follows notification settings, tags configuration and final review.

So, how does it all play together, and how to test it? The simplest solution is to change something in the application. This way, the user knows that the load balancing is working properly. here I placed 3 different instances in target groups of the ELB.

- instance ID i-0d7a715a3d71a242a of type t2.micro (lower left)
- instance ID i-0006ddabb11107f59 of type t3a.nano (lower right)
- instance ID i-07f642432ea759ef3 of type t3.nano (top) which is part of the autoscaling group and more of these instances will be created upon the higher load on this instance

In the picture below can be seen 3 consecutive requests sent to URL o the ELB. If more requests are sent, they will be spread equally across all of them in this alternating pattern.

More interesting is the possibility of scaling based on load. The test will be executed on

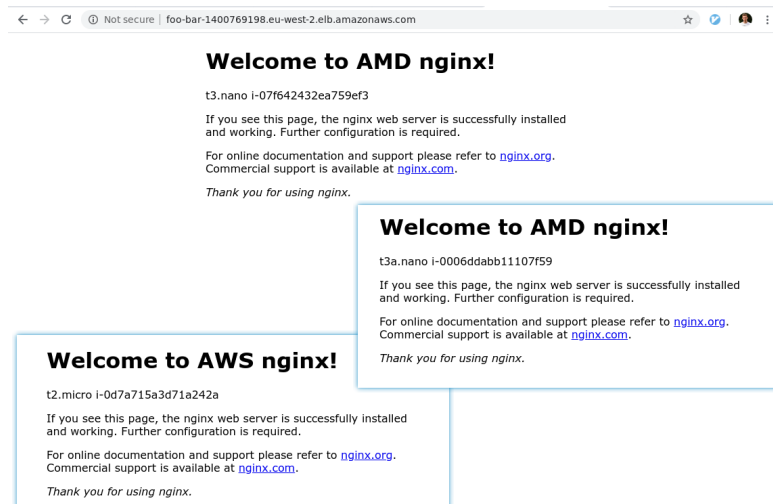


Figure 1.15: ELB testing (screenshot from 29.4.2020, source: author)

instance i-07f642432ea759ef3 as it is part of the ASG.

In this test, stress-testing will be executed with application `stress`. This application has many options to use, more info available in `man stress`. For this use-case will be suitable following configuration, as it allows to use up all capacity of the 2 CPU cores which t3.nano offers: `stress --cpu 2 --io 2 --vm 1`. In `htop`, application for visualization of hardware usage of resources, and also which processes use specific amounts of these resources, we can see that the CPU usage increases dramatically.

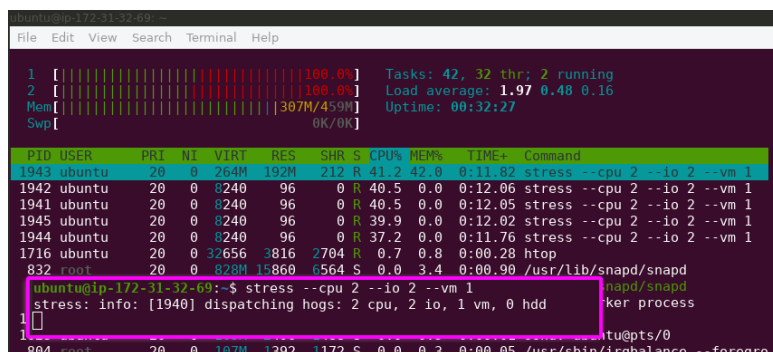


Figure 1.16: ASG stress testing (screenshot from 29.4.2020, source: author)

CloudWatch notices this behaviour, and after crossing a threshold, the alarm changes state from "OK" to "ALARMED".

A command to Auto Scaling is given, handling this increase, after which the instance is started, and when warmup is complete, the instance is being added to the target group.

After this, a stable state is achieved where all instances are healthy.

When no load is present any more, one of the instances is removed from the group, then stopped and terminated. This way, there is no downtime as no request can get to that particular instance.

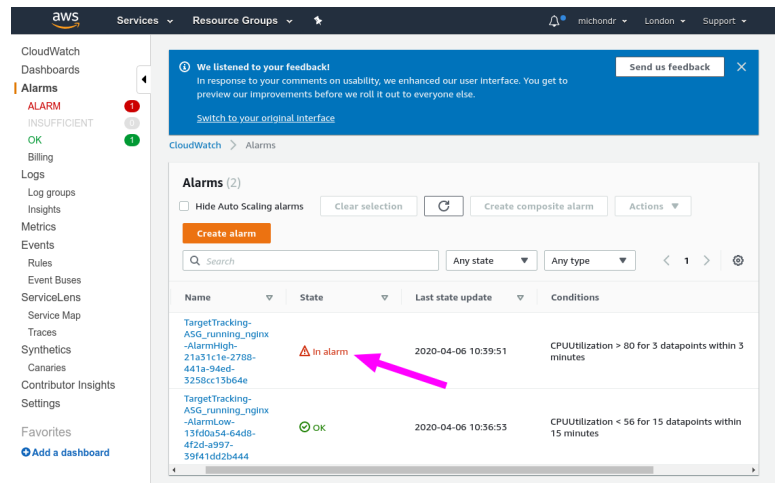


Figure 1.17: CloudWatch alarmed (screenshot from 29.4.2020, source: author)

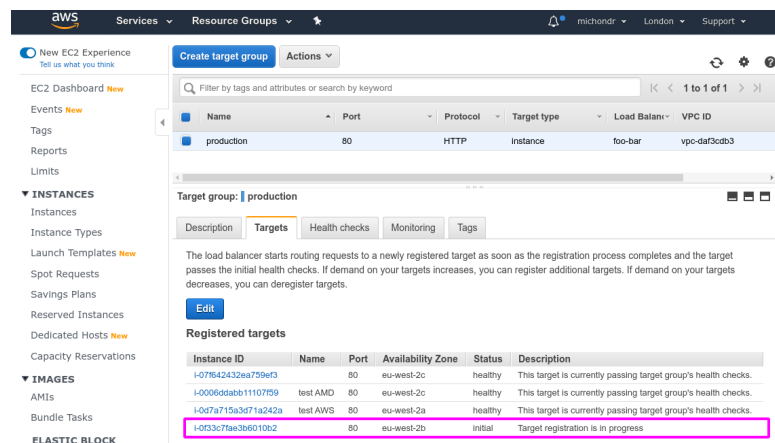


Figure 1.18: Target group instance adding (screenshot from 29.4.2020, source: author)

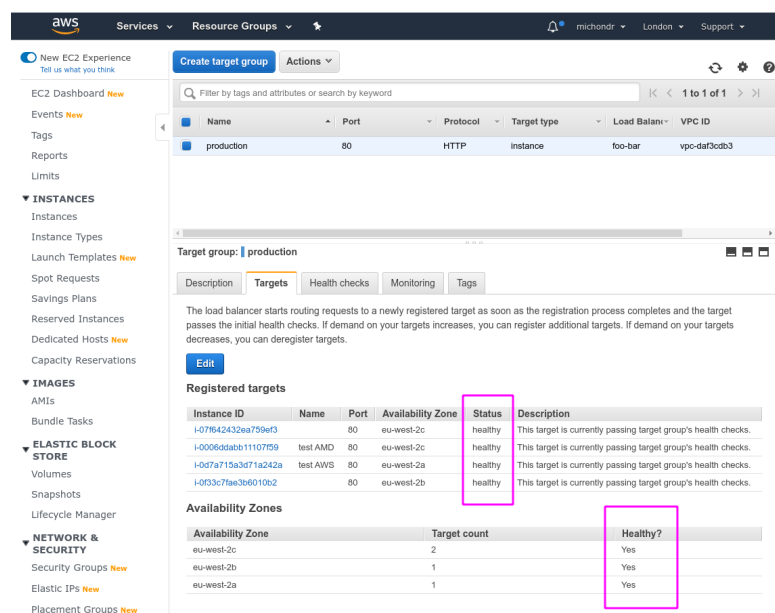


Figure 1.19: Target group healthy state (screenshot from 29.4.2020, source: author)

After all this, the state of the system is stable again in a healthy state.

## EC2 tricks

Scaling up or down either horizontally or vertically takes time and sometimes effort to do so manually. EC2 offers one more trick up its sleeve, and that is boosting of instance hardware resources. Why change the instance or spin up more of them if the environment is virtual and can be edited on-the-fly? That is exactly what T series instance types do.

"T3 instances accumulate CPU credits when a workload is operating below the baseline threshold. Each earned CPU credit provides the T3 instance with the opportunity to burst with the performance of a full CPU core for one minute when needed. T3 instances can burst at any time for as long as required in Unlimited mode." [17]

For further explanation, I'll describe this behaviour on a detailed table of T3 instances.

Table 1.3: T3 instances comparison (data gathered from [aws.amazon.com](https://aws.amazon.com/ec2/instance-types/), May 2020, summarized by the author)

| Name       | vCPUs | Memory (GiB) | Baseline Performance | CPU Credits earned/hr | On-Demand Price/hr |
|------------|-------|--------------|----------------------|-----------------------|--------------------|
| t3.nano    | 2     | 0.5          | 5%                   | 6                     | \$0.0052           |
| t3.micro   | 2     | 1.0          | 10%                  | 12                    | \$0.0104           |
| t3.small   | 2     | 2.0          | 20%                  | 24                    | \$0.0209           |
| t3.medium  | 2     | 4.0          | 20%                  | 24                    | \$0.0418           |
| t3.large   | 2     | 8.0          | 30%                  | 36                    | \$0.0835           |
| t3.xlarge  | 4     | 16.0         | 40%                  | 96                    | \$0.1670           |
| t3.2xlarge | 8     | 32.0         | 40%                  | 192                   | \$0.3341           |

For example, t3.large instance is running with 2 virtual CPU cores and 8GB of RAM. Baseline performance for this instance type is 30%, meaning that if the usage of each of the cores is under 30% while doing some undemanding tasks or idling, CPU credits are earned. These are earned on millisecond-level resolution up to a certain point - the number of credits possible to earn in 24 hours.

When CPU utilization rises above the baseline, credits are being used. One credit provides the performance of a full CPU core running at 100% for one full minute. Or some combination, e.g. two vCPU cores at 50% for one minute or two vCPU cores at 100% for 30 seconds. The same way as earning credits, they are used-up on a millisecond scale while being split on small fractions.

To know when the instance is above the baseline, take a look in CloudWatch on CPU metric, which shows utilization per vCPU. If the value is above 30%, the credits are used, if it is below 30%, credits are being earned. So for t3.large instance, this means that every minute below 30% CPU utilization is worth 0.3 minutes at 100% CPU utilization. Or 0.6 minutes at 50%. If it is critical to keep the instance running at 100% when the load strikes, there is an option to pay an additional charge if a "T2/T3 Unlimited" option is checked while the instance is created.

So is it worth it? It's difficult to make any conclusion without knowledge of the specific use-case. Most probably these instances are not the best option for continuous load, but rather for use-cases where spikes in load are present. AWS itself present these instance for usage in certain kind of scenarios:

"Burstable performance instances are well suited for a wide range of general-purpose applications. Examples include microservices, low-latency interactive applications, small and medium databases, virtual desktops, development, build, and stage environments, code repositories, and product prototypes." [18]

## 1.3 GCP

GCP offers similar features as AWS in means of horizontal and vertical scaling. And that is in terms of accessible VM starting from a disk image, load balancing or autoscaling groups. GCP even allows burstable instances; however, with a different approach, then AWS does.

### 1.3.1 Regions and zones

GCP has similar regional differentiation compared to AWS. They even admit it in their guide "Google Cloud for AWS Professionals":

"Nearly all AWS products are deployed within regions located around the world. Each region comprises a group of data centres that are near each other. Amazon divides regions into availability zones. Similarly, Google Cloud divides its service availability into regions and zones that are located around the world." [19]

However, not all is the same as GCP offers a multi-regional level location for some of their services. Google App Engine and Google Cloud Storage can be set up in such configurations. Current multi-regional locations include Europe, Asia and the United States.

As seen in the picture, the scattering of Google data centres is quite similar to Amazon. This spread is mostly affected by populated places in some regions of the continents.



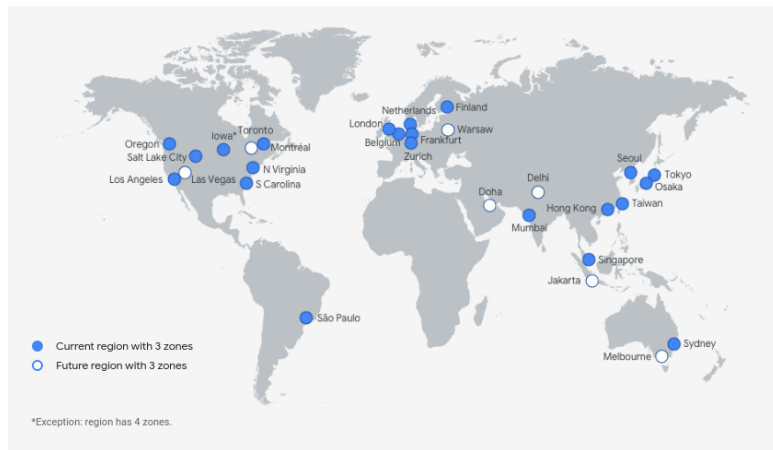


Figure 1.20: GCP regions (source: cloud.google.com, April 2020)

What is more, Google provides a simple visualization of its fibre network. Being the owner of such technology allows Google to use the unimpeded connection for fast and reliable access to all the services provided.

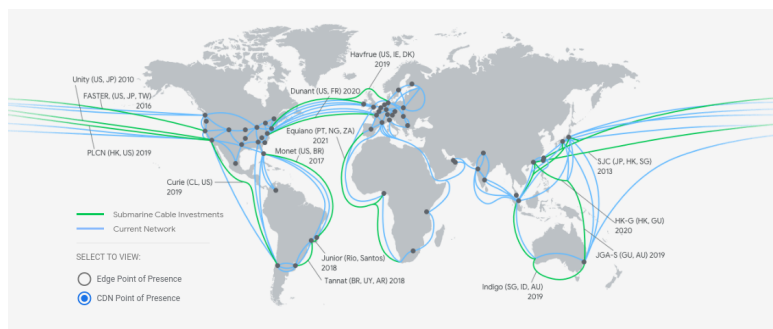


Figure 1.21: GCP network (source: cloud.google.com, April 2020)

### 1.3.2 GCE described

When Amazon released EC2 as their first service in 2006, they went in a direction towards server hosting for their developing customer base. Google choose a different tactic in 2008, as their first released product was the App engine - fully scalable and fully managed solution for application hosting. It was not until 2012 when they released Google Compute Engine [20].

It took one more year before GCE became globally available [21]. GCE essentially works on the same principles as EC2 by AWS.

"Compute Engine delivers configurable virtual machines running in Google's data centres with access to high-performance networking infrastructure and block storage." [5]

## GCE machine types

Same as AWS, GCP offers the user to choose from different types of the virtual machine.

- general-purpose - N1, N2, N2D, E2 types - these are equipped with a balanced amount of vCPU and RAM, great for a variety of workloads
- compute-optimized - C2 - types offering the highest performance per core, optimized for CPU-intensive workloads
- memory-optimized - M1, M2 - offering more memory per virtual core than any other type. Suitable for memory demanding workloads such as in-memory databases.

Apart from this primary difference, there are shared-core instances. That means the instance is running on a fraction of the vCPU and occasionally can burst to full vCPU for short amounts of time (seconds). An in-depth test has been done by OpsDash, who visualized this behaviour quite well. they concluded already in 2017 that f1-micro machine (being the cheapest hardware to run in GCP) is assigned actual 0.2 of a vCPU core, and its CPU usage cannot rise any higher than that [22]

VMs can also be marked as preemptible, meaning the instance is short-lived for less money - the same as AWS offers spot instances. Unlike AWS, which allows these instances to run until they choose differently, GCP has a more strict approach, and that is a maximum of 24 hours of live time.

## Spinning up a GCE VM

the VM can be started in two ways

- using gcloud application - via command,  
e.g. `gcloud compute instances create testing-instance --machine-type=f1-micro --preemptible`
- using GCP console

Following part shows how easy it is to start a VM in GCP console, the web interface.

Firstly, the user must go to "Compute Engine" and "VM instances". Then click on "create instance".

What is the biggest strong-point of GCP is the ease of use - from to start the machine is on a single page, so the process is much simpler, more comfortable and possibly even faster.

Even the boot disk image selection is way more natural to navigate - clean OS selection is a two-step process:

- select operating system (Microsoft Windows Server, Linux distros or few more specific options)
- select version (On Ubuntu there are up-to-date versions and even minimal versions)

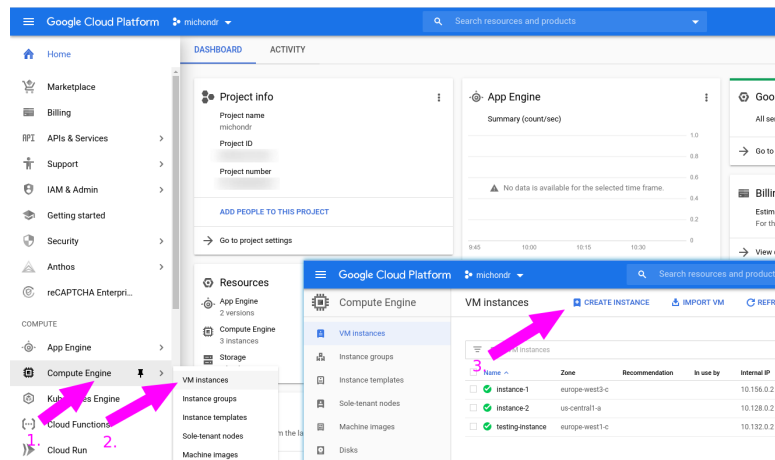


Figure 1.22: GCE machine creation navigation (screenshot from 30.4.2020, source: author)

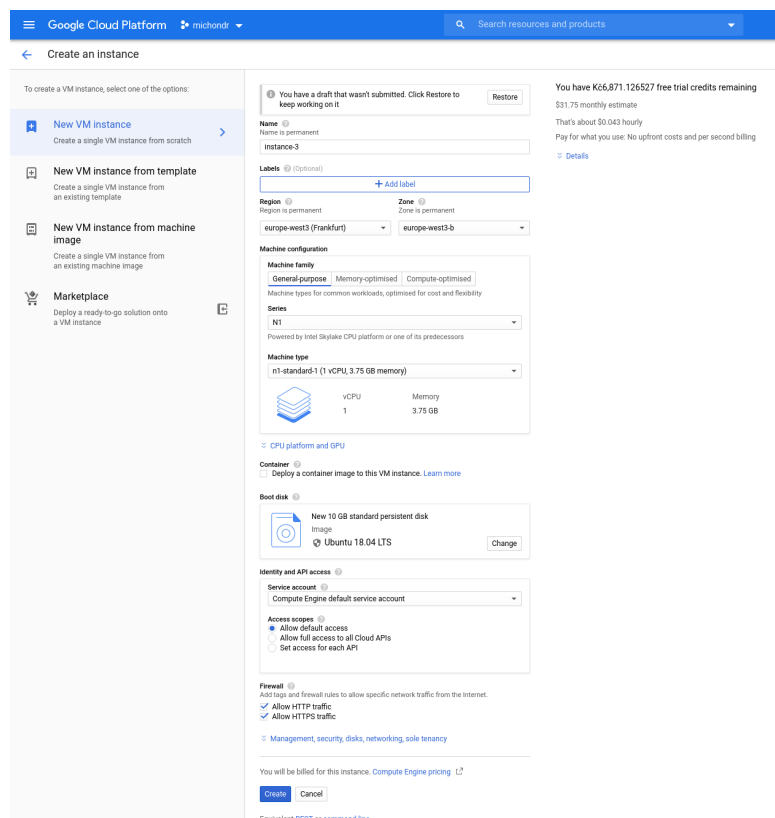


Figure 1.23: GCE create machine (screenshot from 30.4.2020, source: author)

Selection of a custom image or an existing disk is a matter of course.

Another small benefit is an in-browser ability to SSH to the instance. When VM is started, simple instance administration can be done by connecting to the machine straight in the browser. Upon clicking one button, the connection to the instance is established with a one-time SSH key pair.

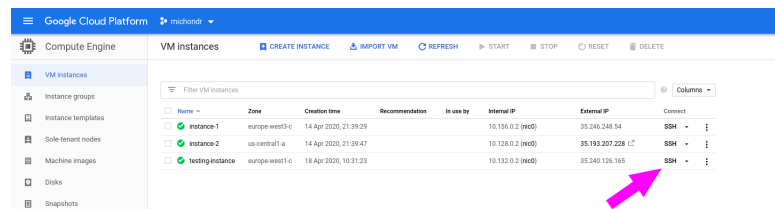


Figure 1.24: SSH to GCE VM (screenshot from 30.4.2020, source: author)

### 1.3.3 Scaling

GCP has few tricks up to its sleeves regarding scaling and the ease of use. The process is, however, mostly similar to AWS. Because of that, no new concepts are described here, as they are mentioned in-depth in part about AWS.

#### Vertical scaling

Vertical scaling can be done similarly by creating a machine image of the VM and then creating a new, bigger machine with this image.

Another option is to stop the instance, detach its disk and start a new instance with the old disk.

However, few things might change in the future to ease up this critical scaling. Google introduced the technology of Live Migration, ensuring that many critical failures do not result in a catastrophic downtime.

We might see technology similar to this in the future where user schedules such migration, including switching to a different sized VM.

#### Horizontal scaling

Here is shown how easy it is to set up a load balancer in GCP, including testing of such scaling. Usage is similar to the AWS example, therefore HTTP request balancing.

Firstly, a new Instance Group must be created. This service is practically the same concept as Autoscaling Groups - there is a template chosen, and a set of rules in place, which create

or terminate instances based on the load of the whole system. These rules can even be static, meaning that instead of creating manually many VMs, an image can be created, stored and then all the VMs can be started at once.

Further information can be found in GCP documentation where all the concepts and terminology is described in detail [23]

Secondly, the user must navigate to load balancer creation and follow the creation process.

The user must configure the backend and the frontend - this way the load balancer knows where to receive requests from and where to target them.

User is presented with four steps to finish the configuration.

1. Backend configuration
2. Host and Path rules
3. Frontend configuration
4. Review and finalization

In the backend creation the Instance Group is selected, based on the instance template. This is the selection of group of instances which is automatically scaled based on the set metrics.

Following is the frontend creation. User can specify one or multiple endpoints where the load balancer is available. One can be allocated to HTTP communication, one for HTTPS/HTTP2 and possibly another for some specific company-internal communication.

The HTTPS frontend needs a certificate for successful configuration. This certificate can be a custom uploaded one or a new one, google generated.

Last but not least, the progress is finalized, and upon clicking "Create", the load balancer is created.

In the detail of the LB, there are two essential pieces of information. One is the summary of frontends with generated IPs, a single frontend with IP 34.107.140.176 on port 80 in this case. The other one is a health check status on the backends. In this situation, we can see that two of the three started instances are healthy and allocated to respond to requests.

When testing this IP in the browser, clearly the requests are divided in between the instances as shown in the figure 1.27 bellow. Upon calling IP of the LB on 34.107.140.176 the VMs of the backend are rotated. The internet protocol address shown in the Nginx testing pages is pasted in statically the HTML file on the VM. This file is being target in web-server configuration.

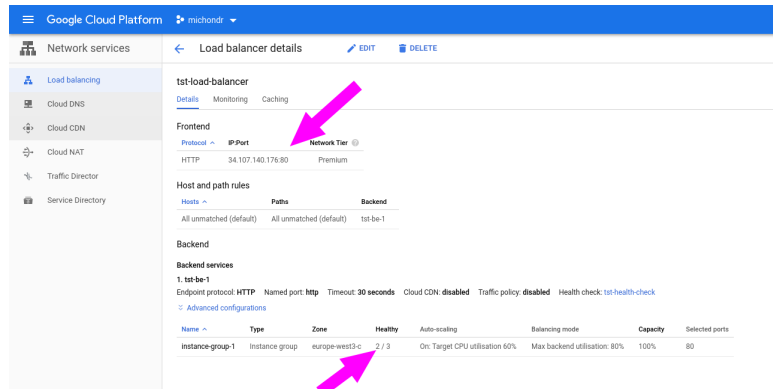


Figure 1.25: GCP load balancer test (screenshot from 30.4.2020, source: author)

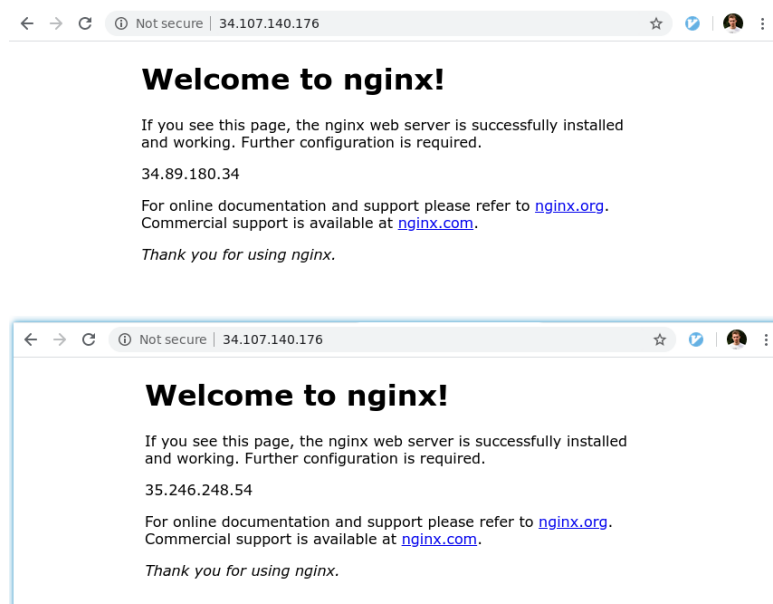


Figure 1.26: GCP load balancer detail (screenshot from 30.4.2020, source: author)

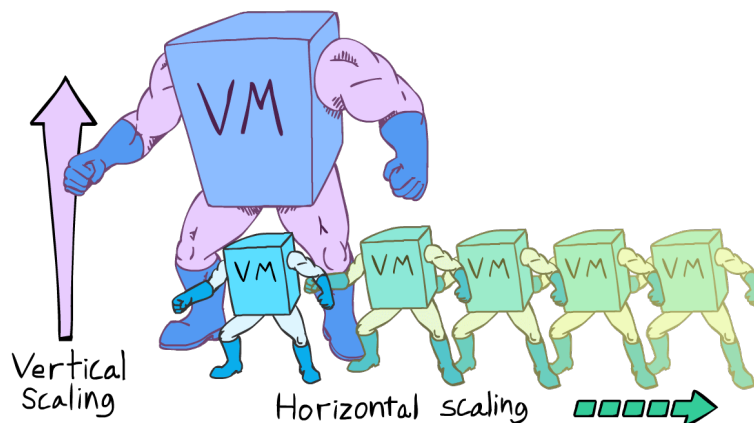


Figure 1.27: vertical vs horizontal scaling (source: turbonomic.com, June 2017)

## 1.4 Conclusion

Comparison of horizontal and vertical scaling depends highly on the use-case and possibilities of the application running. If the application can run on multiple instances, and there is an option to set up autoscaling behind a load balancer, then horizontal scaling is a viable option for improvement in server demands. It might also improve the cost of running the system, as the least amount of instances is running at any given time.

Vertical scaling might be useful in scenarios where the demand for computational power changes for more extended periods. It is possible to scale up and down quickly because of the ease of doing so. The root device disk still exists; only the instance is bigger or smaller.

And usage of burstable instances is something that should be tried, as it might suit the needs of a specific application.

Amazon and Google have many similarities and are not differentiating in many cases. Most significant being design of the web interface, it's UX and logical flow of actions. Price is also a factor, as many times, GCP is cheaper than AWS.

To conclude how virtual servers are obtainable, based on the approach of navigating through the web interfaces, they can be started and terminated very fast and easily by following intuitive steps.

Every user is attracted to one solution or the other based on the approach of the provider, the whole ecosystem and pricing of consumed service.





## 2. Simple file sharing across platforms

General user needs to transfer files between computers or mobile devices. People can either share a file via email, Facebook or Google Drive. All of these has some drawbacks. Cloud services might have a solution to simplify this process.

Reason for this chapter is to understand AWS S3, top-rated service of Amazon Web Services. S3 has been one of the main pillars of why people know AWS - but it is not the only service of this kind. The central part contains a comparison of competition in the means of Google Cloud Platform's service Simple Storage.

These services are mainly used to store files like documents and images to be accessible during development and production use of applications. However, they can be used for simple file sharing, because that is precisely what many people do in their life day by day. People share resources at work or school, and people share pictures of their recent trips and many more.

### 2.1 Methodology

Firstly it is essential to define existing commonly used solutions, find their benefits and shortcomings. Then cloud services are introduced, addressing some of these issues and offering solutions that can substitute these commonly used solutions.

Many important topics are covered, including setting up such storage, usage of tools to upload and download from this storage and of course, pricing of said storage.

The goal is to find out which of covered solutions is a better choice for simple file sharing, and choosing provider based on a number of factors. Primary factors being price and secondary factors being the size of the userbase and benefits resulting from this

#### 2.1.1 Common approach to share files

These are options which most people use. They are the easiest ones where nothing can break; no setup is needed, as these solutions simply work.

In the following part, I go in detail of positives, negatives, usages and shortcomings of many ways files can be shared using common approaches - USB drive, Facebook Messenger and Email

## USB drive

This approach might feel like being in a stone age these days, but offline transfer might be still the fastest way of sharing for many people. The transfer is limited only in size and in speed by the quality of the flash storage.

When it comes to big files, the transfer from one device to the USB drive and then out of it, the required time to finish this operation can be fatiguing, especially on the older USB 2.0 drives with theoretical maximal speed of 53 MB/s [24]. User must copy/move the file on the USB, then move somewhere either in the room or across the world, then copy/move the file from the USB.

Until some time ago, this had been faster than the internet. Not anymore, as internet speeds are reaching 25MB/s upload and download. On average 5MB/s upload 10MB/s download in 2020 [25].

## Facebook Messenger

Messenger is vast, one of the biggest communicators, very popular in the Czech Republic. In 2019, it was used worldwide by 1.3 billion people [26]. Facebook is a huge company, which means they need to take extra precautions to files you can transfer. Transferred pictures and videos are compressed, and on the rest of the files, there is 25MB limit [27].

What is interesting is how Messenger handles pictures which are wide or tall. A file was compressed to 10.1% of its original capacity with very much unreadable contents - possibly because of more than 95% reduction in pixels.

Table 2.1: Messenger picture compression in wide aspect ratio (summarized by the author)

|            | <b>File raw state</b> | <b>File state after Messenger compression</b> |
|------------|-----------------------|-----------------------------------------------|
| Size       | 2.7MB                 | 267kB                                         |
| Resolution | 1710x10949            | 319x2048                                      |

On pictures with more standard aspect ratio, the compression is not that bad, although the files still lose a significant amount of detail.

Table 2.2: Messenger picture compression in 16x9 ratio (summarized by the author)

|            | <b>File raw state</b> | <b>File state after Messenger compression</b> |
|------------|-----------------------|-----------------------------------------------|
| Size       | 557,5 kB              | 25,6 kB                                       |
| Resolution | 2560x1440             | 853x480                                       |

This image compressions is fine when users see the image on a small screen, however, it is unacceptable for almost anything else

## Email

While SMTP directly does not specify the maximum size of a message, ESMTP enables mail server administrators to filter-out messages with size over some value. This ties up users hands in means of maximum file size to transfer. This size is no less than 10MB [28].

When sending large files over email, the client service might let you know that the file could not be sent.

This is how the notification looks in Gmail. it attaches as a reply to the conversation under the email

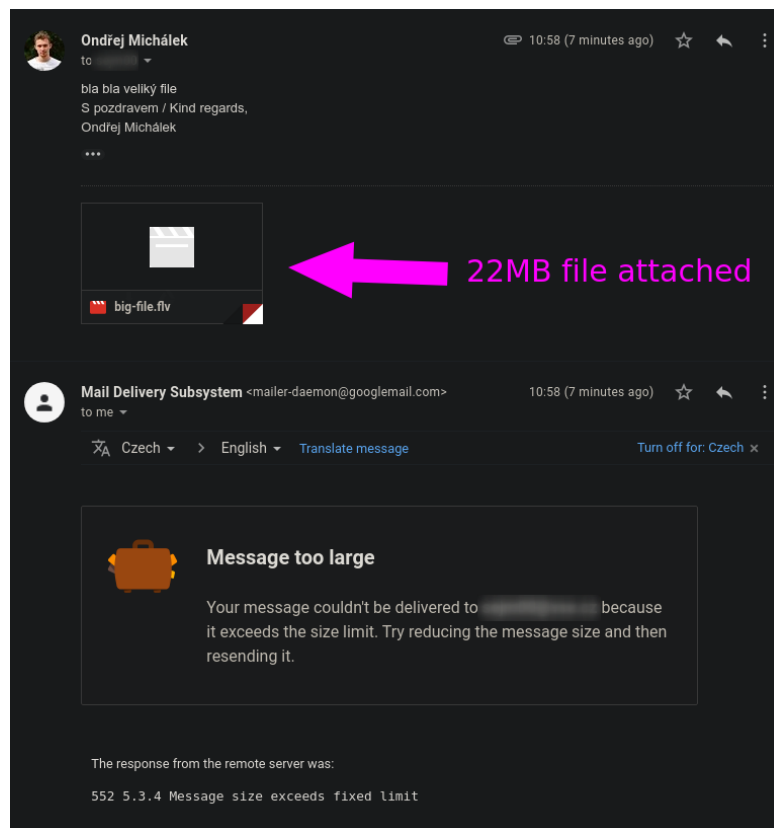


Figure 2.1: Gmail file too large notice (screenshot from 23.3.2020, source: author)

## Google Drive

Google enables users to upload/download all sorts of files, as it works like standard file storage. One particular problem is a problem with some filetypes. For example, SSH keys cannot be uploaded to Google Drive. This might be because of an algorithm which reads contents of a file. This way Google knows what users are uploading, and also to provide the user with full-text searching in file contents

On the contrary, the great benefit of Google Drive it's its convenience to use. Drive has support on mobile phones - Android and Apple - and also on desktops. Users can connect

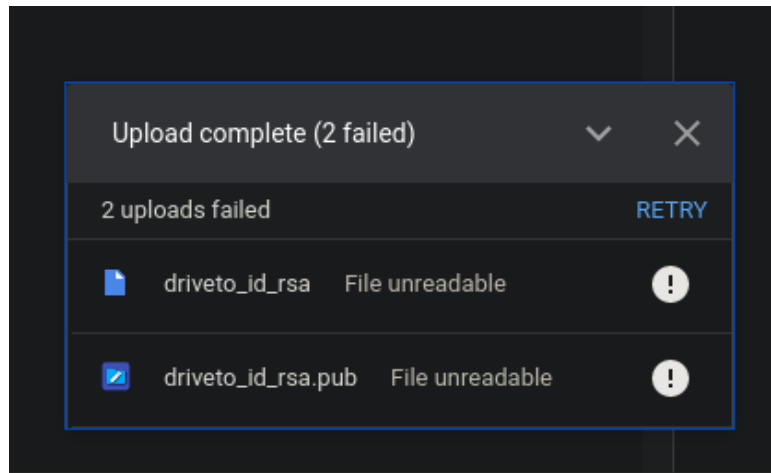


Figure 2.2: Google Drive ssh key upload fail (screenshot from 23.3.2020, source: author)

their Drive account as a folder in Windows; then all the files can be synchronized as they desire

Google delivers all these features to users for free up to 15GB of storage, so if the only use-case is to share files, this is the only limit user has (apart from internet connection speed). Fortunately, users tend to use Drive as a cloud storage because of its convenience. Files can be edited right in Google GSuite applications, so most of them already have set up Google account and are partially in the Google product ecosystem, so sharing files on Drive is the easiest option for them

as previously mentioned, these services are free up to 15G of storage. if more is demanded, user can choose to pick 100GB, 200GB, 2TB, 10TB, 20TB or 30TB package. The first two costs 59.99 CZK per month and 79.99 CZK per month respectively, which might be very affordable for most people

### 2.1.2 Cloud services approach to share files

AWS and GCP offer multiple products for cloud storage. Some are more flexible than others. When the user needs to transfer files, some are more suitable than others.

Storage services offered by GCP [29] :

- Cloud Storage - Reliable object storage with global edge-caching and instant data access
- Persistent disk - High-performance block storage for virtual machines and containers
- Cloud Storage - Ultra-low-cost archival storage with online access speeds
- Cloud Filestore - Fully managed, scalable file storage with predictable performance
- Cloud Storage for Firebase - Scalable storage for user-generated content from your app
- Data Transfer Services - Fast offline, online, or cloud-to-cloud data transfer
- Drive enterprise - Cloud-based content collaboration and storage

Storage services offered by AWS [30] :

- Amazon Elastic Block Store (Amazon EBS) - Persistent local storage for Amazon EC2, for relational and NoSQL databases, data warehousing, enterprise applications, Big Data processing, or backup and recovery
- Amazon Elastic File System (Amazon EFS) - A simple, scalable, elastic file system for Linux-based workloads for use with AWS Cloud services and on-premises resources. It is built to scale on-demand to petabytes without disrupting applications, growing and shrinking its size automatically as you add and remove files, so your applications have the storage they need – when they need it.
- Amazon FSx for Lustre - A fully managed file system that is optimized for compute-intensive workloads, such as high-performance computing, machine learning, and media data processing workflows, and is seamlessly integrated with Amazon S3
- Amazon FSx for Windows File Server - A fully managed native Microsoft Windows file system built on Windows Server so you can easily move your Windows-based applications that require file storage to AWS, including full support for the SMB protocol and Windows NTFS, Active Directory (AD) integration, and Distributed File System (DFS)
- Amazon Simple Storage Service (Amazon S3) - A scalable, durable platform to make data accessible from any Internet location, for user-generated content, active archive, serverless computing, Big Data storage or backup and recovery
- Amazon S3 Glacier & Amazon S3 Glacier Deep Archive - Highly affordable long-term storage classes that can replace tape for archive and regulatory compliance
- AWS Storage Gateway - A hybrid storage cloud augmenting your on-premises environment with Amazon cloud storage, for bursting, tiering or migration
- Cloud Data Migration Services - A portfolio of services to help simplify and accelerate moving data of all types and sizes into and out of the AWS cloud
- AWS Backup - A fully managed backup service that makes it easy to centralize and automate the back up of data across AWS services in the cloud as well as on-premises using the AWS Storage Gateway

From this palette of available solutions are two noticeable, Cloud Storage from GCP and S3 from AWS. Both provide virtually unlimited storage space where management is taken care of by the provider. This means fearless ad-hoc or long-term storage of files in their original shape and form. No downscaling, no compression, no artificial speed limitation.

## 2.2 Comparison of GCP Cloud Storage and AWS S3

Both of these services work on similar principles. They allow the user to upload files and access them, set privileges on files and folders, set availability and location; They even call the base unit of storage a "bucket".

Cloud storage:

Buckets are the basic containers that hold your data. Everything that you store in Cloud Storage must be contained in a bucket. You can use buckets to organize your data and control access to your data, but unlike directories and folders, you cannot nest buckets. Because there are limits to bucket creation and deletion, you should design your storage applications to favour intensive object operations and relatively few buckets operations. [31]

S3:

A bucket is a container for objects stored in Amazon S3. Every object is contained in a bucket. For example, if the object named `photos/puppy.jpg` is stored in the `johnsmith` bucket in the US West (Oregon) Region, then it is addressable using the URL `https://johnsmith.s3.us-west-2.amazonaws.com/photos/puppy.jpg`. [32]

Essentially these buckets work like an online directory. What is the key difference? For every user, something else. Either encryption, geolocation of the physical data, or the pay-as-you-go pricing. They allow the user to upload files either from their web interface or from a variety of sources using their SDK and client libraries. What is meant by that? Everybody can create a simple bash command to upload a file or create complex logic in a web application where is need for files management. The essential part is that the bucket name must be unique, because it is part of the buckets absolute URL. This works similarly as domain names.

### 2.2.1 Process of bucket creation

Currently is the process of bucket creation straightforward on both providers. Following table shows how providers approach it in their interface.

GCP Cloud Storage:

1. Name your bucket - Choose the unique name of your bucket
2. Choose where to store your data - Allows user to specify the physical location of the data in Google's data centres
  - Region - This options stores data in a single data centre. It contributes to lower price and lower latency and also lower availability. A specific location is chosen from many data centres across America, Europe and Asia
  - Multi-region - Data is stored in multiple data centres. It comes for a higher cost, provides higher availability. The user chooses continent as location
  - Dual-region - Specifically chosen two data centres which offer low latency and high availability. Options are America (Iowa and South Carolina) and Europe (Netherlands and Finland)
3. Choose a default storage class for your data - GCP allows user to decide how often he accesses the data, and lower the price if the frequency is low enough. This can lower

the storage price per GB/month to 20% while data retrieval cost jumps from none to 0.05 GB/month. GCP offers few baselines user can choose from, which balance storage cost and data retrieval price

- Standard - Best for short-term storage and frequently accessed data
  - Nearline - Best for backups and data accessed less than once a month
  - Coldline - Best for disaster recovery and data accessed less than once a quarter
  - Archive - Best for long-term digital preservation of data accessed less than once a year
4. Choose how to control access to objects
    - Fine-grained - User can specifically set permissions on separate objects in the bucket
    - Uniform - Global bucket permissions apply for each file in it. This is suitable for example while the bucket is used as a container for static website hosting
  5. Advanced settings
    - Encryption - Google sets encrypt the bucket by default, user can choose a google-managed key or upload his own
    - Retention policy - Minimum amount of time between file is uploaded, and file can be deleted. Number and unit is required (seconds, days, months, years)
    - Labels - An organization is everything, labelling system is crucial for some users

AWS S3:

1. Name and region
  - Bucket name - This must be unique in all the buckets AWS has
  - Region - Select from a variety of locations in US, Europe and Asia
  - Copy settings from an existing bucket - A handy feature which enables users to simply copy all the settings instead of painstakingly set them all over again
2. Configure options
  - Versioning - This option is disabled by default, and enables you to keep all versions of an object. That means that when uploading a new object with the same name, the object is overwriting the previous object, but the previous versions is still available. When deleting an object, instead of removing it from the bucket, a delete marker is inserted. This request results in 404 when accessing the object, but when version ID is specified, the object can be still fetched. Some amount of programming can achieve this feature, but many people benefit from the fact that it is built-in and tested by AWS
  - Server access logging - S3 has the feature to show the owner which requests were sent to access the bucket. This might be a great information source for understanding S3 bill or in case of security audits.
  - Object-level logging - This is essentially step-up of the Server access logging, it allows to have more information about accessing the data in the bucket
  - Tags - The same principle as Labels in Cloud Storage
  - Default encryption - Enables AES-256 or AWS-KMS server-side encryption of the uploaded objects at the moment the data are about to be written. This however

- does not help with security during data transfer
  - Object lock - This is the same as Retention policy in Cloud Storage, where you can prevent the object from being overwritten or deleted indefinitely or for the specific amount of time
  - CloudWatch request metrics - Enable request monitoring, similar to server-level logging, but instead of saving logs into a different S3 bucket, the metrics are shown in graphs in the CloudWatch
3. Set permissions
    - User can specify how visible the bucket is to the rest of the world. AWS chooses to set permissions strictly by default, as all public access is blocked. Necessary access can be further allowed for usage of the bucket
  4. Review
    - All previous steps are reviewed on a single page, marking the existing progress

As we see, GCP and AWS have many features common. GCP Cloud Storage seems to be more progressive in terms of the core data storage, and it enables users to choose the regions and the expected storage time. AWS has different service for long-term archival storage called S3 Glacier

AWS S3 pushes their service more towards data security, salvaging information about how the data is transferred and who transfers them. This might be highly beneficial for data management.

Both services allow fine settings in access and security. Both achieves this by ACL (Access Control List) permissions.

## 2.2.2 How can users use this?

It is great to have access to data storage service, but the user must have easy access to upload and download files. This approach is different on each platform, as users typically use different operating systems - Windows, Mac, GNU/Linux, Android and iOS will all need a different approach. Usually, there are already built tools to accommodate this need:

- Mountain Duck allows seamless S3 and Cloud Storage integration on Windows and Mac
- S3 Explorer is available on Android Play Store
- AWS S3 Manager is available on Apple App Store

Unfortunately, it seems not many people make clients for GCP Cloud Storage for mobile devices, possibly because of the lower popularity of Cloud Storage compared to S3. There is a library for accessing Cloud Storage from Android, but that means developing a custom application. It is not a ready-to-use client.

Where the majority of people use these services are people operating servers, running GNU/Linux. Majority of production servers run Unix - to be specific about 70.2% of websites [33]. It is no



surprise that Amazon and Google made efforts so that their libraries run on these systems.

## Uploading files to S3

Apart from various libraries for specific languages, AWS provides also a tool for CLI.

The AWS Command Line Interface (CLI) is a unified tool to manage your AWS services. With just one tool to download and configure, you can control multiple AWS services from the command line and automate them through scripts. The AWS CLI introduces a new set of simple file commands for efficient file transfers to and from Amazon S3.

`aws-cli` is not only for S3. it handles many of the AWS services such as EC2, SNS, SQS, RDS, route53, IAM, Glacier, Kafka, Lambda and many more. (for the full list see `aws help`)

This tool is familiar for unix users, as it uses most of typical commands as `ls`, `cp` or `rm`.

- to list contents of a bucket `aws s3 ls s3://bucket_name/`
- to upload file to a bucket `aws s3 cp file_name s3://bucket_name/ --acl public-read`
- to remove file from a bucket `aws s3 rm s3://bucket_name/file_name`

The `--acl public-read` option in copy command is specified so that the file can be public from anywhere on the web. Without it, any user is greeted with access denied response

```
<?xml version="1.0" encoding="UTF-8"?>
<Error>
  <Code>AccessDenied</Code>
  <Message>Access Denied</Message>
  <RequestId>4F0111EE1651B7C4</RequestId>
  <HostId>4rSXWt+jLD0KtKQV1Fpzx/gXu0hZutuJ/3m2Q8=</HostId>
</Error>
```

`aws s3` allows file URL to be pre-signed instead of making it open to the world. That means if the user wants to share the file for a certain period of time, it is possible to generate a pre-signed URL which contains temporary access key and expiration information in the URL on behalf of the user.

- unsigned URL example `https://bucket_name.s3.amazonaws.com/file_name`
- signed URL example `https://bucket_name.s3.amazonaws.com/file_name?AWSAccessKeyId=key&Signature=sig&Expires=1555531131`

`aws-cli` can work flawlessly with multiple profiles and users. Each of them must have a record in `~/.aws/credentials` on the local machine, then it is merely about specifying the profile: `aws --profile=profile-name ....` this way different profiles can be specified, so that constrained access is used only for the features and services that are currently needed.

Accessing files from public URL is easy as well; they are located in `https://bucket_name.s3.amazonaws.com/file_name`. If all the security tasks are handled correctly, such as setting correct

ACL rules on the bucket, the file is available.

Scripting can be introduced to make sharing files more convenient.

```
s3cp () {  
    if [[ $2 == '--public' ]]; then  
        aws --profile=s3cp s3 cp $1 s3://bucket_name/ --acl public-read  
        echo https://bucket_name.s3.amazonaws.com/$(basename $1)  
    else  
        aws --profile=s3cp s3 cp $1 s3://bucket_name/  
        aws --profile=s3cp s3 presign s3://bucket_name/$(basename $1)  
    fi  
}
```

This simple bash function can be put into `.bashrc` or any other location where it can be executed. usage:

- `s3cp file_name`. File is uploaded to s3, presigned for default of 60 minutes and the signed URL printed in terminal.
- `s3cp file_name --public`. File is uploaded to s3 and ACL is set to public-read. File is indefinitely public on the web and its URL is printed in terminal.

The presigned expiration time can be set with `--expires-in=number_of_seconds` [34].

## Uploading files to Simple Service

GCP provides many libraries for the most used languages the same way as AWS does. These include Python, Java, PHP and many more [35].

Apart from these, GCP provides `gcloud` CLI interface, and even a `gcloud` interactive based around the SDK. How-to-install instructions are available in Guides.

Same as `awscli`, `gcloud` has a similar command to manage files:

- to list contents of a bucket `gsutil ls gs://bucket_name/`
- to upload file to a bucket `gsutil cp -a public-read file_name gs://bucket_name`
- to remove file from a bucket `gsutil rm gs://bucket_name/file_name`

GCP also allows pre-signed URLs, but only with obtained Service Account key [36].

When a user goes to URL of an uploaded file which has its permissions not set correctly, GCP tries to acquire users log-in through OAuth. There is no XML response about the file not being accessible. This URL to access file is in the following format: `https://storage.cloud.google.com/bucket_name/file_name`.

Scripting can be introduced to make upload more convenient.

```
sscp () {
```

```

if [[ $2 == '--public' ]]; then
    gsutil cp -a public-read $1 gs://sdasds/
    echo https://storage.cloud.google.com/sdasds/$(basename $1)
else
    gsutil cp $1 gs://sdasds/
    gsutil signurl private_key_file gs://sdasds/$(basename $1)
fi
}

```

Executing this function can be done in two ways, similar to the previous example:

- `sscp file_name` uploads file to GCP Simple Storage and pre-signs its access URL. The output of pre-signing is printed in terminal
- `sscp file_name --public` file is uploaded to S3 and ACL is set to public-read. The file is indefinitely public on the web. this link is printed in terminal

### 2.2.3 Pricing

Both AWS and GCP offer similar services in terms of file storage. Interesting is their pricing strategies. Pricing is collected in March 2020, AWS region eu-central-1 and GCP europe-west3. both of these are in Frankfurt

- storage (GiB-month)
  - AWS - \$0.0245
  - GCP - \$0.023
- data retrieval (GiB-month)
  - AWS - \$0
  - GCP - \$0
- creation requests (AWS calls this "PUT, COPY, POST, LIST requests", GCP calls this "class A operations") per 1000 operations
  - AWS - \$0.0054
  - GCP - \$0.005
- fetch requests (AWS calls this "GET, SELECT, and all other requests", GCP calls this "class B operations") per 1000 operations
  - AWS - \$0.00043
  - GCP - \$0.0004

This pricing by any means not final, as it varies on the place of data upload, place of data download, place of data storage, storage options (sharding, multiple region storage, access frequency,...)

GSP allows user more transparent view, as they show the user price calculations based on his selection throughout the bucket creation process. This is a very user-friendly feature.

All the data is shown on the right side of the picture. Each of these inputs can be filled in

Create a Bucket

1

Name your bucket

Pick a globally unique, permanent name. [Naming guidelines](#)

bucket\_name\_123

Tip: Don't include any sensitive information

CONTINUE

Choose where to store your data

This permanent choice defines the geographic placement of your data and affects cost, performance and availability. [Learn more](#)

Location type

☒ Region

Lowest latency within a single region

☐ Multi-region

Highest availability across largest area

☐ Dual-region

High availability and low latency across 2 regions

Location

us-east1 (South Carolina)

CONTINUE

Choose a default storage class for your data

A storage class sets costs for storage, retrieval and operations. Pick a default storage class based on how long you plan to store your data and how often it will be accessed. [Learn more](#)

☒ Standard

Best for short-term storage and frequently accessed data

☐ Nearline

Best for backups and data accessed less than once a month

☐ Coldline

Best for disaster recovery and data accessed less than once a quarter

☐ Archive

Best for long-term digital preservation of data accessed less than once a year

CONTINUE

Monthly cost estimate

Enter values below to check this bucket's monthly cost. For guidance only. [Pricing details](#)

Storage and retrieval

Storage size

GB

\$0.020 per GB-month

Data retrieval size

GB

Free

Operations

Class A operations

per month

\$0.005 per 1,000 ops

Class B operations

per month

\$0.0004 per 1,000 ops

Availability SLA: 99.9%

Monthly cost: \$0.00

Currency: US Dollar (\$) ▼

Figure 2.3: GCP bucket creation pricing calculation (screenshot from 23.3.2020, source: author)

for estimations, and the expected monthly cost is calculated.

## 2.3 Conclusion

Amazon Web Services Simple Storage Service and the Google Cloud Platform Simple Storage are both very similar services, with similar usage, tools, pricing and even terminology. GCP is in this market with a great disadvantage, as AWS is very popular, and the S3 service is an excellent example of that.

S3 has more than 20x times more followers based on data from stackshare [37] resulting in wider user-base, resulting in more supporting libraries, resulting in more spread community of people with hands-on experience, resulting in more people using it. This also results in benefits coming hand-in-hand, such as third-party clients.

However, GCP's documentation is extensive, clear to understand with multiple examples of usage. To take the decision into extremes where the amount of data stored/transferred results in significant price difference, then the GCP Simple Storage is a better solution over AWS S3.

Nonetheless, both of these services are beneficial for the end-user if his work environment can be adapted for the usage of `gsutil` or `aws-cli` libraries and his skill in the shell allows such

60

usage. For many reasons - ease-of-use, configurability to best-fit user needs and last but not least, affordability.



## 3. Identity and Access Management

One of the supporting services provided is Access Management. In general, this is the place where new users are created and deleted, their permissions added and revoked. This is one of the most critical management tools, as this service ensures that users and applications have wide enough scope of privileges that they can operate services they work with, and narrow enough that security is not at stake.

IAM (Identity and Access Management) services are free of charge on both of the providers, AWS and GCP. They are a management tool for all the other services provided as security is essential - the importance of it differs based on the situation.

A single person trying out the services and looking around in the console does not care that much about security and about granular settings for every service he tries. However, customer large or small, paying for provided services and with a business where an immense amount of money is at stake is cautious about the security and granular access to each used service.

### 3.1 Methodology

Interfaces of both providers are introduced, analyzed and then compared. The objective is to find out which of the providers have more features available in the structure of IAM differs significantly and pinpoint any of the differences.

As mentioned in previous chapters, AWS and GCP have strong similarities in terms of functionality. It is essential to be aware if there are any shortcomings in tool managing security, as it might result in a potentially hazardous situation for one of the providers. If there are, it could be one of the factors contributing to choosing one provider over the other.

The analysis is based around features provided and granularity of privileges that can be assigned users based on service.

### 3.2 AWS

AWS Identity and Access Management (IAM) is a web service that helps you securely control access to AWS resources. You use IAM to control who is authenticated (signed in) and authorized (has permissions) to use resources. [38]

Amazon pushes users to think about security from the first time they log in to the console. When a person creates an account, he becomes the root user. This user has unlimited access in the console, as he is the owner of the account. This user possesses the root credentials that can be used to login and to do everyday tasks. However AWS themselves advise not to do so.

When you first create an AWS account, you begin with a single sign-in identity that has complete access to all AWS services and resources in the account. This identity is called the AWS account root user and is accessed by signing in with the email address and password that you used to create the account. We strongly recommend that you do not use the root user for your everyday tasks, even the administrative ones. [38]

Best practice scenario is then to create the first IAM user and lock away the root credentials. Both types of users can log in to the AWS Console easily through a simple form:

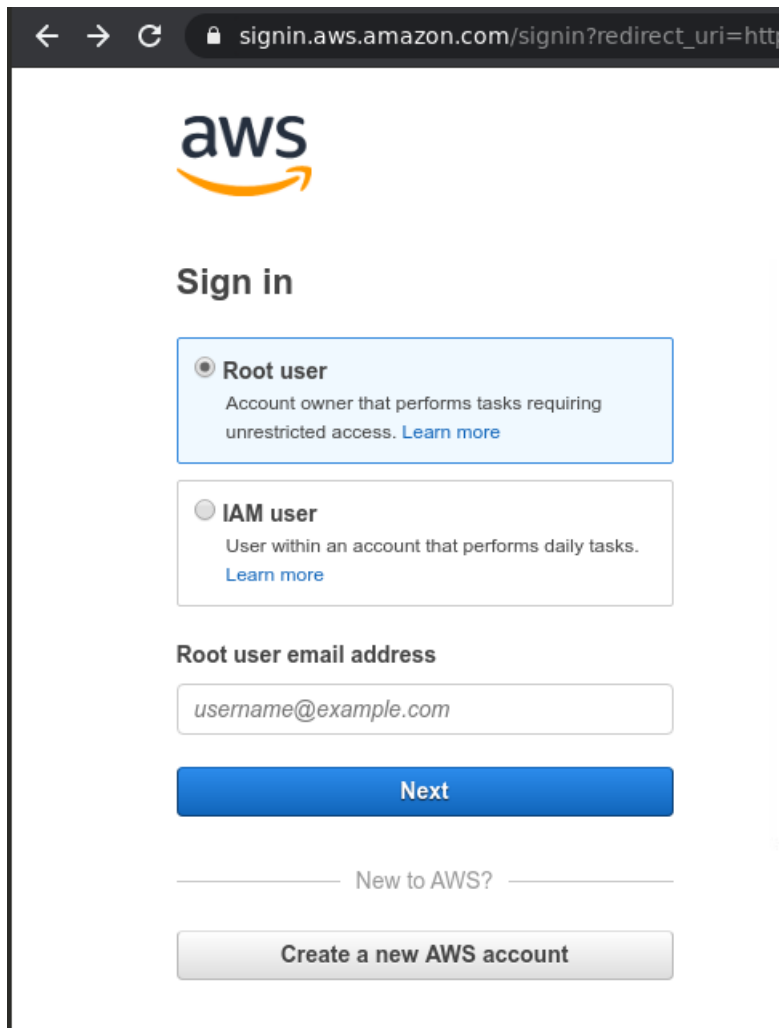
A screenshot of the AWS sign-in page. The browser's address bar shows the URL 'signin.aws.amazon.com/signin?redirect\_uri=http'. The page features the AWS logo at the top, followed by the heading 'Sign in'. Below this, there are two radio button options: 'Root user' (selected) and 'IAM user'. The 'Root user' option includes a description: 'Account owner that performs tasks requiring unrestricted access. Learn more'. The 'IAM user' option includes a description: 'User within an account that performs daily tasks. Learn more'. Below these options is a text input field labeled 'Root user email address' containing the placeholder text 'username@example.com'. A blue 'Next' button is positioned below the email field. At the bottom of the form, there is a link 'New to AWS?' and a button 'Create a new AWS account'.

Figure 3.1: AWS login form (screenshot from 26.4.2020, source: author)

This section is where the more advanced users are divided from the novice ones, as this default login screen should never be used if possible for two reasons:

1. root credentials are critical and should be used rarely
2. the option of logging in through the IAM user only asks the AWS account ID, which is a 12 digit number, which redirects the user to URL `https://account-id-here.signin.aws.amazon.com/console` where the user is asked his username and password.



The IAM login screen located in the second argument should be the access point of all users logging in.

### 3.2.1 Access management

AWS has multiple logical units which have distinct characteristics. The core unit is the Users unit - every person or application with access to AWS services should have their user account.

When every person has separated access to AWS, all the benefits of granular access management became evident. Also, credentials leakage impact is mitigated, and the person is responsible only for his/her access. Also, the user can be given access to the console (the web interface), the CLI, or both. This implies that users are created for programmatic access to AWS.

When programmatic access is given, the user is provided with an "access key ID", and a "secret access key". These two values are used to authenticate while using the AWS API, `aws-cli` or an SDK. When the AWS Management Console access is given, the user is provided with a password (autogenerated or custom, possible to require password reset) that allows him access to the AWS console.

Another logical unit is Groups. When multiple users require equal access, they all can be members of a group and have their access attached based on the said group. Example of this might be a team of DevOps - it is possible that for their work they need high access to EC2. It is straightforward to attach `FullAccess` privileges on a DevOps group, and then add each of the people to this group. Some of the users can have attached more access privileges on an individual basis.

Apart from Groups and Users, there are Roles. These allow granting permissions to multiple targets:

- AWS services - Allows AWS services to perform actions on user behalf. These include roles for Autoscaling Groups which start and terminate EC2 instances, or Lambda functions uploading files to S3
- Another AWS account - Example could be that a different company partially builds a project, and employees of that company must set up the environment in your project
- Web identity - Allows users federated by the specified external web identity or OpenID Connect (OIDC) provider to assume this role to perform actions in the account
- SAML 2.0 - Allows users that are federated with SAML 2.0 to assume this role to perform actions in the account

And most importantly, the policies, which group specific access levels to specific resources. Policies can be custom made or selected, as Amazon offers predefined policies for simplicity and ease of use. These predefined policies are for every of 228 services AWS provides in at least two versions - `FullAccess` and a `ReadOnlyAccess`. Example of these are the predefined

policies for S3 seen in the picture below:

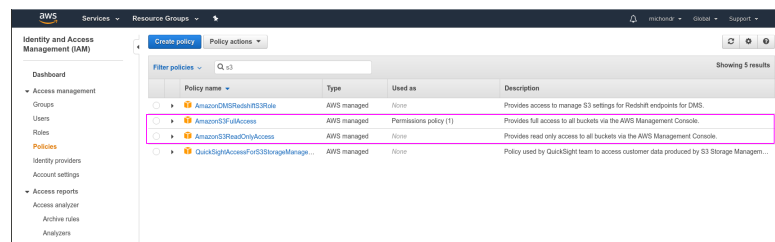


Figure 3.2: AWS IAM policy for S3 - FullAccess and ReadOnlyAccess (screenshot from 26.4.2020, source: author)

Policies can be custom made to suit the needs of one process, as they can be attached to specific roles or users. But even more importantly, they should be specific to a single usage.

As an example, let's go in detail about a policy suited for use in access to S3 as described in chapter "Simple file sharing across platforms". The policy is restricted as much as possible for reasonable comfort of usage. This all could have been skipped by assigning the user with AmazonS3FullAccess policy, but that would be unnecessary. Even more, allowing the user to change bucket policies, change object versions, and bucket deletion would be unreasonable.

Because of this, the policy is given access only to S3 service and the necessary object actions assigned.

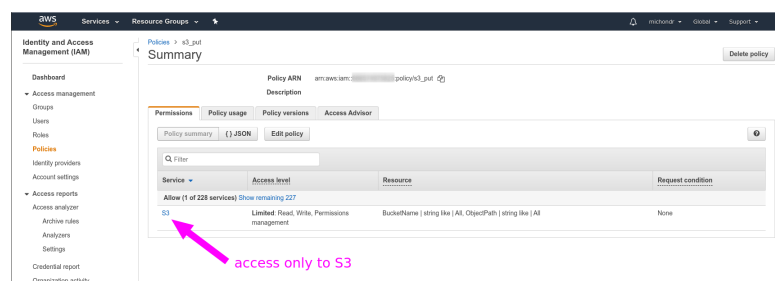


Figure 3.3: AWS s3\_put policy - list of services (screenshot from 26.4.2020, source: author)

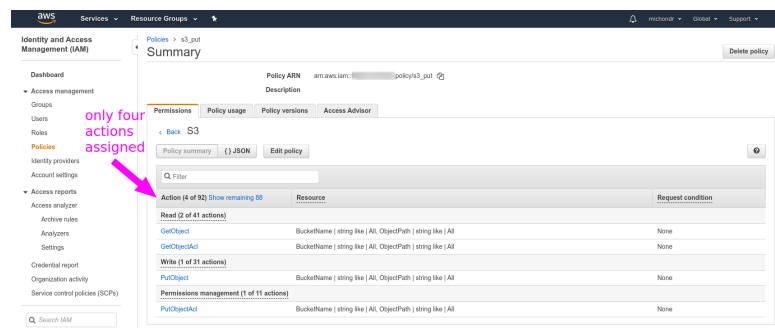


Figure 3.4: AWS s3\_put policy - list of actions (screenshot from 26.4.2020, source: author)

Policies, Groups and users can be easily extended, narrowed and deleted later on. This offers great flexibility for every change that occurs.

## 3.3 GCP

"Cloud Identity and Access Management (Cloud IAM) enables you to create and manage permissions for Google Cloud resources. Cloud IAM unifies access control for Google Cloud services into a single system and presents a consistent set of operations." [39]

There are significant differences in project ownership in GCP. The project is not designed to be owned by a single person (with root account access), but rather as a community project. A single person created the project under his profile, and this person is enrolled as the owner, however, more users can be added and have this exact role. This way, the ownership of the project can be easily and safely transferred.

Google decided to be more specific in the approach of dividing logical units of access - holders of such access are the Members. They possess permissions added through roles.

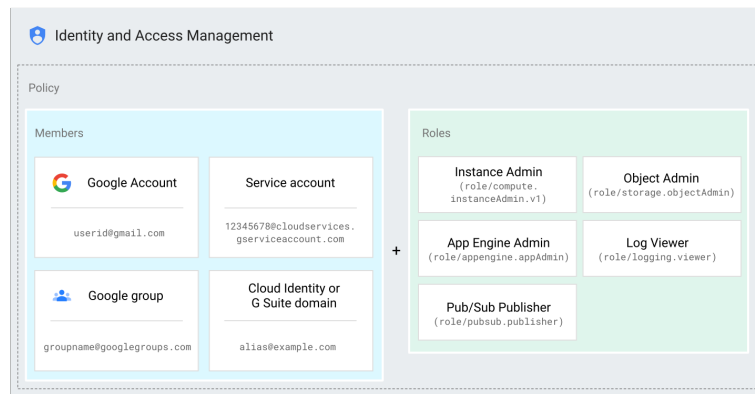


Figure 3.5: GCP IAM (screenshot from 26.4.2020, source: author)

### 3.3.1 Members

Members are the identity holding the access and the permission. It can be one of the following:

- a user with a Google Account (which most people already have)
- a Google group - containing more than one user
- a Cloud identity or a G Suite domain - G Suite domains are usually registered by companies so that everybody has an email address on the same domain and can use other services of the Google Suite
- a Service Account

First three types specify a person to have the access. A user receives an invitation in his email inbox, and then is authenticated via Google Sign-in

Service Accounts are however made for different usage. These are for programmatical access in an application in cooperation with an SDK. Service Account is firstly given a name in

the format of `name@project-name.iam.gserviceaccount.com`, then assigned desired roles. In the last step, the administrator of this role can be specified, including the specification of the user who can work on behalf of this Service Account.

### 3.3.2 Roles

A role is a collection of permissions which determine what operations are allowed on a resource.

Names of these permissions are in very readable format of `service.resource.action`, e.g. `bigquery.datasets.create`.

What is very unusual in role management is staging - role can be set with a stage based on how precisely is it matching the specific needs. The available selections of stages are as follows:

- Alpha
- Beta
- General Availability
- Disabled

While the creation of a new role, one or more existing roles can be selected in the list, and the permissions of such roles are automatically pre-selected. User can extend or narrow these to specifically suit the needs.

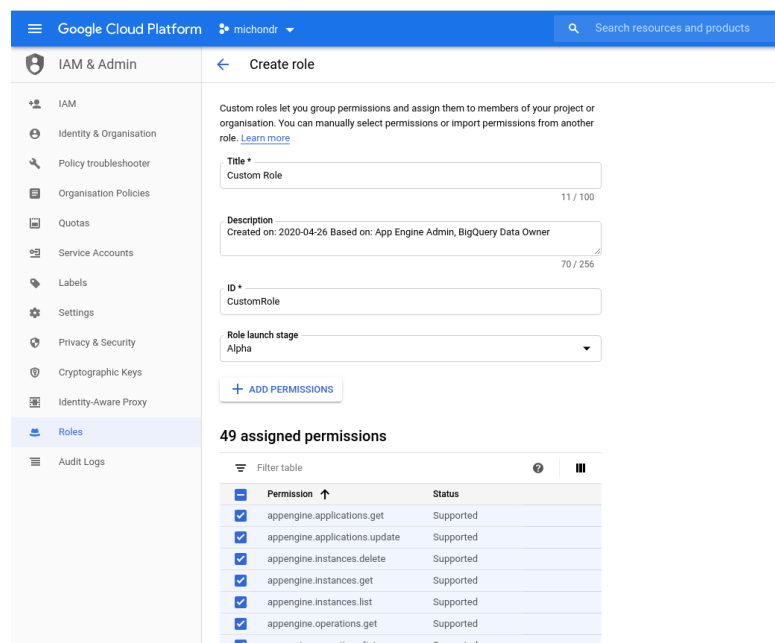


Figure 3.6: GCP role creation (screenshot from 26.4.2020, source: author)

Apart from users and service accounts, GCP provides a few more sections in IAM. One of them is "Quotas". Here the user has complete insight into how many resources are being used

and to what extent. At the time of writing, there are 1646 quotas monitored.

This section is not tightly connected to user/access management. However, it is an important management tool - an example being that quotas contain the count of Service Accounts. In a scenario where this number would be depleted, no more Service Accounts can be created

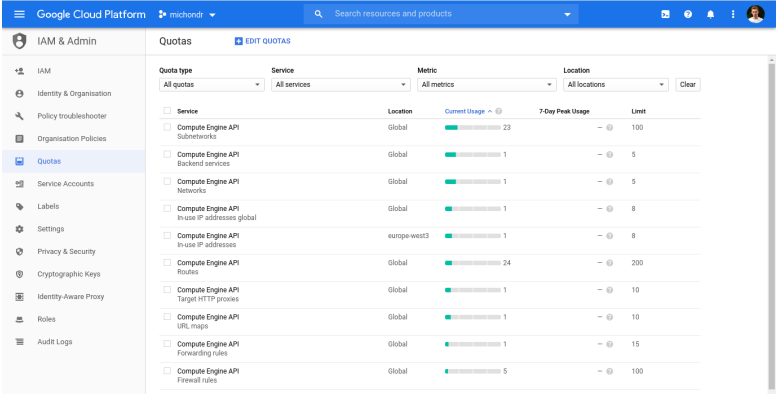


Figure 3.7: GCP quotas (screenshot from 26.4.2020, source: author)

### 3.4 Conclusion

Descriptive introduction of IAM environments of AWS and GCP was given in this chapter. The ease of use provided by both of these providers proves that security management is easier than ever and should not be overlooked. Flexible optimizations can be made to reflect customer needs to the state of the business.

AWS and GCP respectively offer customization of privileges assigned to identities (users) of the projects and accounts. Both providers enable the customer to set the user access very granularly to the level of service feature while enabling grouping this granularity into bigger units.

Little knowledge gained in the research is that the Roles of Service Accounts in GCP should be editable, even the documentation points that out [40], however, the detail page of the Service Account shows no further information. The only option here is to use a simple command to fetch this information using the console gcloud application [41]

Apart from this unfortunate finding o GCP, both providers allow very detailed and granular access assignment and easy user management.



# Conclusion

The Cloud service market continues to proliferate, and new features and services are released regularly. At the same time, the same services are consumed by thousands of customers around the world, large and small. The market always evolves and brings customers new benefits sourced from service evolution and rivalry in the competitive environment.

Moving parts of the infrastructure to the cloud brings some advantages and disadvantages, and every potential customer has his attitude to such a shift. The market is typical by awareness of Amazon Web services, who were the first company providing these services on a worldwide scale and continues doing so to this day. Google Cloud platform is showing in the cloud market with narrow userbase and is catching up with the tempo that Amazon keeps for many years.

The main goal of this work was to compare these two providers, their services and the approach they have on the customer. This has been done throughout three topics within this thesis

- Server scaling of virtual machines (chapter 1)
- File sharing across platforms using simple storage services (chapter 2)
- Identity and Access Management (chapter 3)

The output of this comparative analysis is visible since the beginning of the work, and that is a massive similarity in the services provided. For most customers, no prominent characteristics are shifting their priority towards one or the other.

As many users use the provided web interface, the most distinguishing factor is the user interface and user experience of using it. AWS is clearly aiming for robustness and feature-rich environment; however this overflow of steps needed to complete even a simple task might be uncomfortable for some users in the long run. Google has developed a more lightweight, modern and flat-looking interface which is easier to navigate visually and logically.

What is more important is the price - both providers use the pay-as-you-go model of compensation for the features consumed. Google must be aware of their placement on the market, as they do offer services for a lower price than what does Amazon ask for similar service counterpart. Absolute price comparison is essential in larger quantities of consumed services, which might be one of the arguments to choose GCP over AWS, as this difference is substantial on an immense amount of money spent.

A different result is in approach comparison - Amazon inclines towards demanding users who often don't mind the higher costs. Rule of paying for the premium might be applicable here, as usually customer does not precisely know how much will the service cost before the invoice is sent to him. Google feels more user friendly, as it provides the user with cost information throughout most of the service configurations and even has a section which information about

project resource monitoring, which is something that AWS lacks.

While this work is mostly dedicated to general analysis and examples, it should be said that further and deeper analysis of services provided might be necessary to acquire a deeper understanding of differences between these two providers. What is also necessary for that is the knowledge of actual customers and especially those who used both providers and can give concrete feedback on why they settled on one provider or the other.

In general, it should be said that neither of Amazon AWS or Google GCP is a terrible choice for sole customer, a startup or a bigger company. What sells one over the other is usually bias of the people using the tools and implementing the services. Both of these providers allow free tier usage; it is beneficial to try the services and the environments and choose what suits better either the user or the use-case.



# List of references

1. BALA, Raj; GILL, Bob; SMITH, Dennis; WRIGHT, David. *Magic Quadrant for Cloud Infrastructure as a Service, Worldwide*. Available also from: <https://www.gartner.com/doc/reprints?id=1-1CMAPXN0&ct=190709>.
2. *March 2020 Web Server Survey*. Netcraft, 2020. Available also from: <https://news.netcraft.com/archives/2020/03/20/march-2020-web-server-survey.html>.
3. *Amazon EC2 Instance Types*. Available also from: <https://aws.amazon.com/ec2/instance-types/>.
4. *Easy Amazon EC2 Instance Comparison*. Available also from: <https://ec2instances.info/>.
5. *Compute Engine: Virtual Machines (VMs)*. Available also from: <https://cloud.google.com/compute>.
6. *Machine types*. Available also from: <https://cloud.google.com/compute/docs/machine-types>.
7. *Regions, Availability Zones, and Local Zones*. Available also from: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-regions-availability-zones.html>.
8. *Amazon EC2 Pricing*. Available also from: <https://aws.amazon.com/ec2/pricing/on-demand/>.
9. *Amazon EC2 Spot Instances*. Available also from: <https://aws.amazon.com/ec2/spot/>.
10. *Amazon EC2 Reserved Instances*. Available also from: <https://aws.amazon.com/ec2/pricing/reserved-instances/>.
11. *New – EC2 Spot Instance Termination Notices*. Available also from: <https://aws.amazon.com/blogs/aws/new-ec2-spot-instance-termination-notice/>.
12. *Amazon Linux AMI - what distro is this based on?* Available also from: <https://forums.aws.amazon.com/thread.jspa?threadID=51647>.
13. *EC2 Network Performance Cheat Sheet*. Available also from: <https://dzone.com/articles/ec2-network-performance-cheat-sheet>.
14. *Launch Instance Wizard*. Available also from: <https://eu-west-2.console.aws.amazon.com/ec2/v2/home?region=eu-west-2#LaunchInstanceWizard:>.
15. *Amazon EBS volume types*. Available also from: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-volume-types.html>.
16. *Launch Templates*. Available also from: <https://docs.aws.amazon.com/autoscaling/ec2/userguide/LaunchTemplates.html>.
17. *Amazon EC2 Instance Types - General Purpose*. Available also from: [https://aws.amazon.com/ec2/instance-types/#General\\_Purpose](https://aws.amazon.com/ec2/instance-types/#General_Purpose).

18. *Burstable performance instances*. Available also from: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/burstable-performance-instances.html>.
19. *Google Cloud for AWS Professionals*. Available also from: <https://cloud.google.com/docs/compare/aws>.
20. *Google I/O 2012 - Introducing Google Compute Engine*. Available also from: [https://www.youtube.com/watch?v=0-sF5ZWB\\_FY](https://www.youtube.com/watch?v=0-sF5ZWB_FY).
21. *Google Compute Engine (GCE) becomes Generally Available*. Available also from: [https://yourstory.com/2013/12/google-compute-engine-gce?utm\\_pageloadtype=scroll](https://yourstory.com/2013/12/google-compute-engine-gce?utm_pageloadtype=scroll).
22. *Exploring the Google Cloud f1-micro Instance - OpsDash*. Available also from: <https://www.opsdash.com/blog/google-cloud-f1-micro.html>.
23. *Autoscaling groups of instances*. Available also from: <https://cloud.google.com/compute/docs/autoscaler/>.
24. *Common USB Development Mistakes – You Don't Have to Make Them All Yourself!* Cypress Semiconductor, 2006. Available also from: <https://www.cypress.com/file/88486/download>.
25. *Speedtest Global Index*. 2020. Available also from: <https://www.speedtest.net/global-index>.
26. *Global social media ranking 2019*. 2020. Available also from: <https://www.statista.com/statistics/272014/global-social-networks-ranked-by-number-of-users/>.
27. *Messaging - Messenger Platform - Documentation - Facebook for Developers*. Available also from: [https://developers.facebook.com/docs/messenger-platform/send-messages#sending\\_attachments](https://developers.facebook.com/docs/messenger-platform/send-messages#sending_attachments).
28. *Maximum email size limit for Gmail, Outlook.com, etc.* Available also from: <https://www.outlook-apps.com/maximum-email-size/>.
29. *Cloud storage products*. Available also from: <https://cloud.google.com/products/storage/>.
30. *Cloud Storage with AWS*. Available also from: <https://aws.amazon.com/products/storage/>.
31. *Key terms - Buckets*. Available also from: <https://cloud.google.com/storage/docs/key-terms#buckets>.
32. *Introduction to Amazon S3*. Available also from: [https://docs.amazonaws.cn/en\\_us/AmazonS3/latest/dev/Introduction.html](https://docs.amazonaws.cn/en_us/AmazonS3/latest/dev/Introduction.html).
33. *Usage statistics of operating systems for websites*. 2020. Available also from: [https://w3techs.com/technologies/overview/operating\\_system](https://w3techs.com/technologies/overview/operating_system).
34. *presign - AWS CLI 1.18.51 Command Reference*. Available also from: <https://docs.aws.amazon.com/cli/latest/reference/s3/presign.html>.
35. *Google Cloud Client Libraries*. Available also from: <https://cloud.google.com/apis/docs/cloud-client-libraries>.

36. *Signed URLs*. Available also from: <https://cloud.google.com/storage/docs/access-control/signed-urls>.
37. *Amazon S3 vs Google Cloud Storage*. 2019. Available also from: <https://stackshare.io/stackups/amazon-s3-vs-google-cloud-storage>.
38. *What Is IAM?* Available also from: <https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html>.
39. *Cloud Identity and Access Management documentation*. Available also from: <https://cloud.google.com/iam/docs>.
40. *Granting roles to a service account for specific resources*. Available also from: [https://cloud.google.com/iam/docs/granting-roles-to-service-accounts#granting\\_access\\_to\\_a\\_service\\_account\\_for\\_a\\_resource](https://cloud.google.com/iam/docs/granting-roles-to-service-accounts#granting_access_to_a_service_account_for_a_resource).
41. *How do I list the roles associated with a gcp service account?* Available also from: <https://stackoverflow.com/questions/47006062/how-do-i-list-the-roles-associated-with-a-gcp-service-account>.



# **Annexes**



# A. Navigation of instance management in AWS

In this annex more detailed screenshots are added, visualizing the user experience of navigating in the AWS EC2 Instance launch wizzard.

EC2 can be found in service drop-down on the top of the page

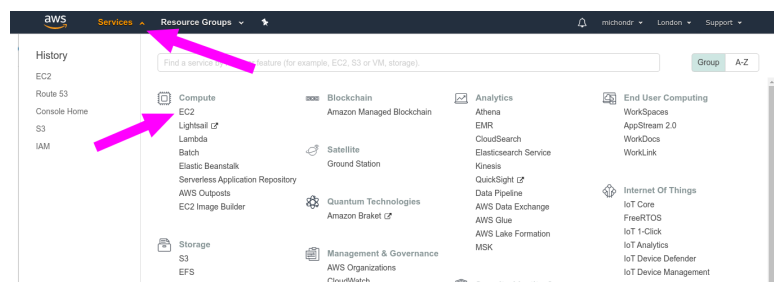


Figure A.1: EC2 where to find (screenshot from 30.3.2020, source: author)

Then on the main page select "running instances"

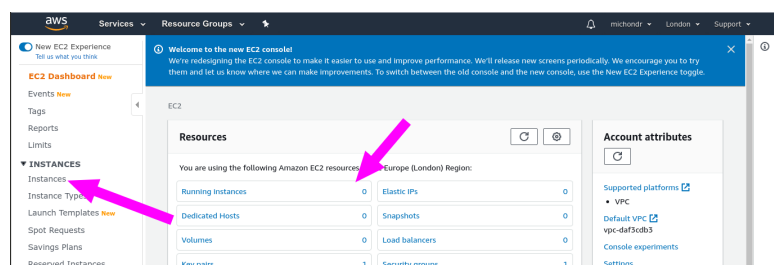


Figure A.2: EC2 dashboard (screenshot from 30.3.2020, source: author)

This link leads users to instances dashboard, where "launch instance" button is present

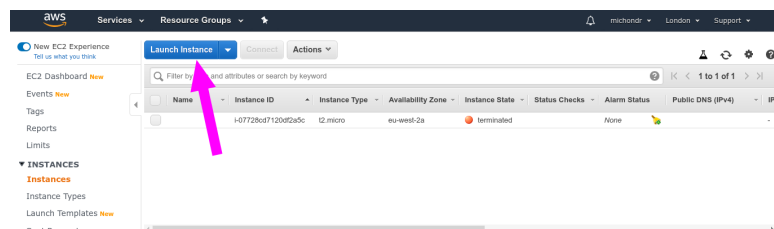


Figure A.3: EC2 launch instance (screenshot from 30.3.2020, source: author)

EBS location

The device will be preselected, so the user clicks on "Actions" and "Create Snapshot".

After filling in the description, click on "Create Snapshot" is necessary.

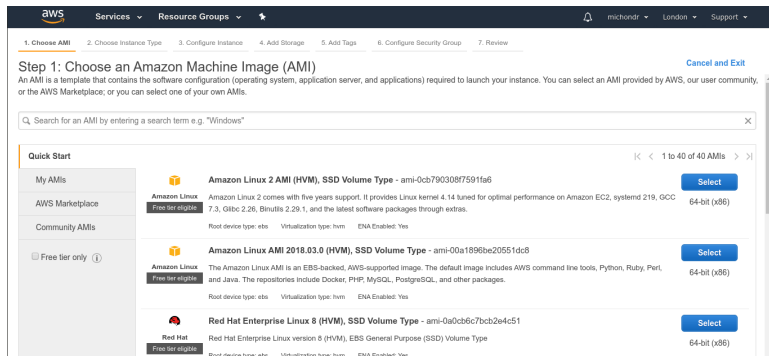


Figure A.4: EC2 select AMI (screenshot from 30.3.2020, source: author)

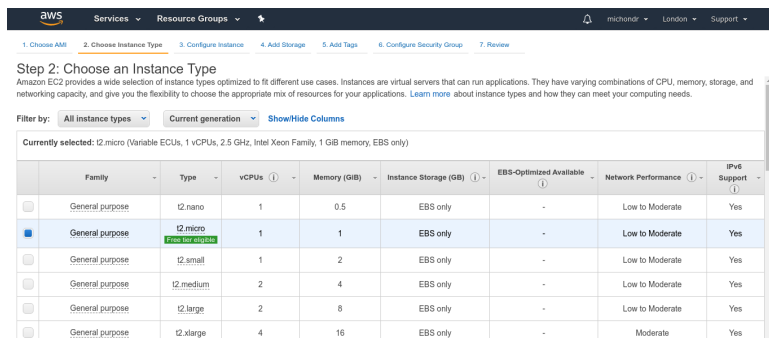


Figure A.5: EC2 select instance type (screenshot from 30.3.2020, source: author)

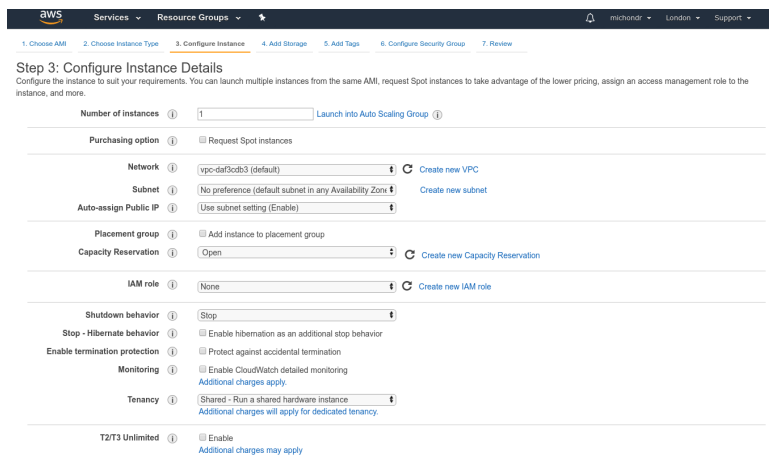


Figure A.6: EC2 configure instance (screenshot from 30.3.2020, source: author)

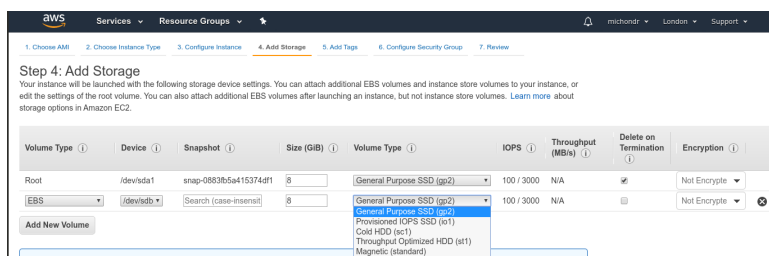


Figure A.7: EC2 add storage (screenshot from 30.3.2020, source: author)



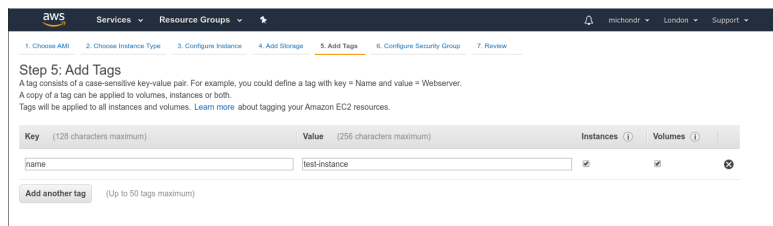


Figure A.8: EC2 add tags (screenshot from 30.3.2020, source: author)

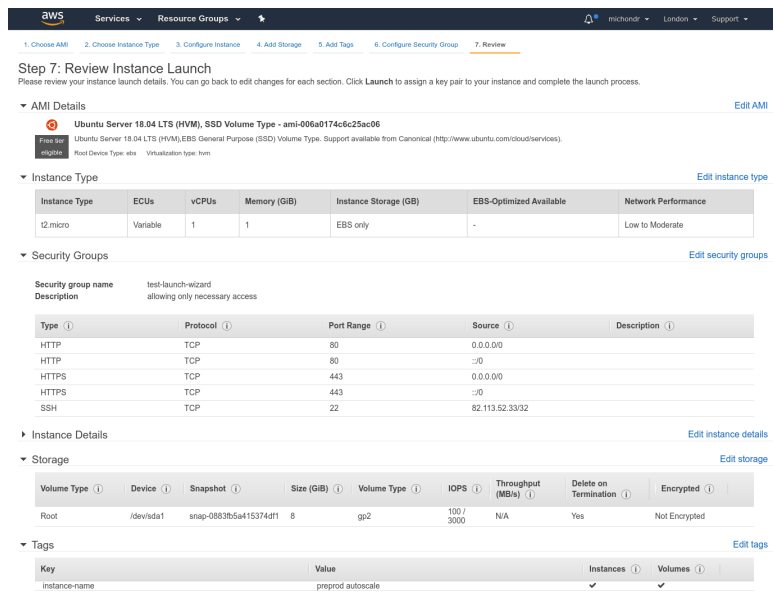


Figure A.9: EC2 review (screenshot from 30.3.2020, source: author)

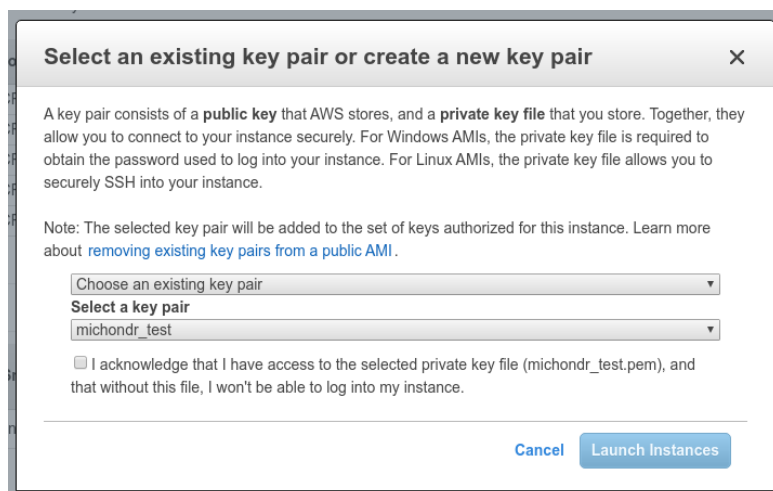


Figure A.10: EC2 SSH key selection (screenshot from 30.3.2020, source: author)

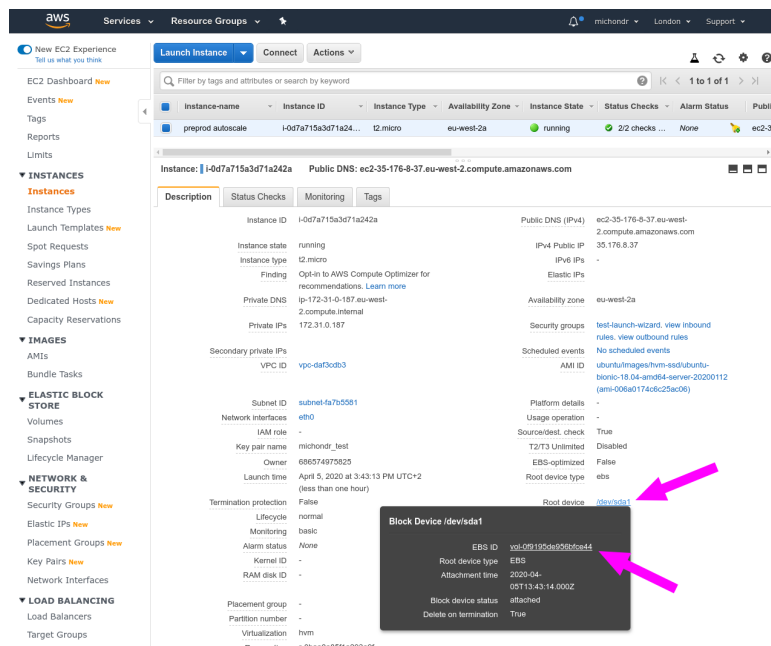


Figure A.11: locating the EBS root device (screenshot from 30.3.2020, source: author)

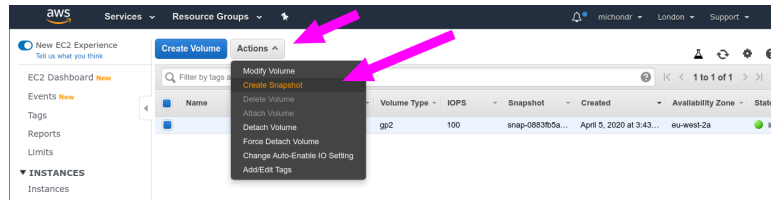


Figure A.12: EBS create snapshot (screenshot from 30.3.2020, source: author)

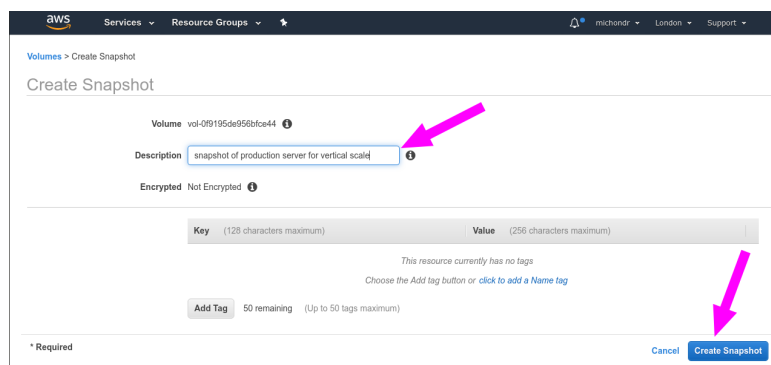


Figure A.13: EBS snapshot creation (screenshot from 30.3.2020, source: author)

In "Snapshots" selection, the created snapshot must be located, selected, and the next step is to select "Create Image" in "Actions".

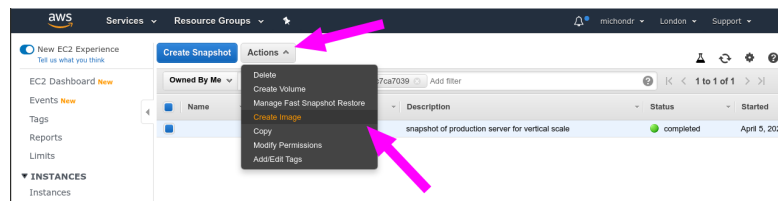


Figure A.14: Snapshot image creation (screenshot from 30.3.2020, source: author)

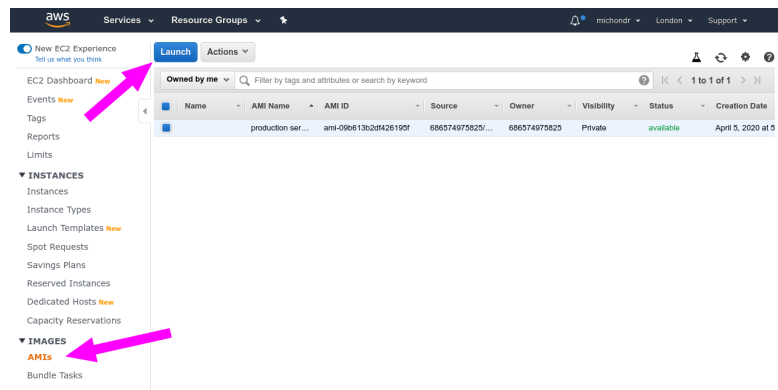


Figure A.15: AMI launch (screenshot from 30.3.2020, source: author)

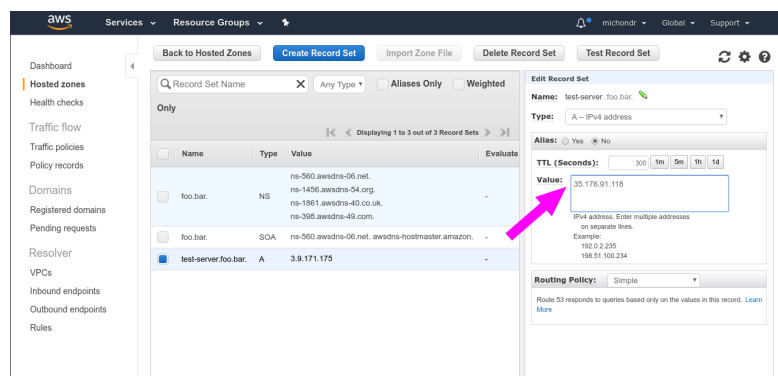


Figure A.16: EC2 instance IP in route53 (screenshot from 30.3.2020, source: author)

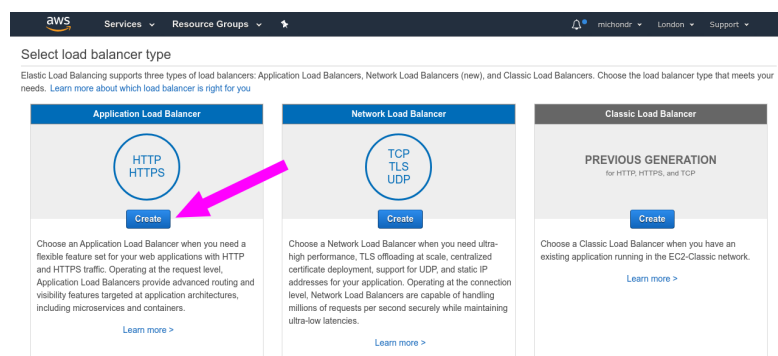


Figure A.17: ELB type selection (screenshot from 30.3.2020, source: author)

**Step 1: Configure Load Balancer**

**Basic Configuration**

To configure your load balancer, provide a name, select a scheme, specify one or more listeners, and select a network. The default configuration is an internet-facing load balancer in the selected network with a listener that receives HTTP traffic on port 80.

**Name** (i)

**Scheme** (i) ☒ internet-facing  
☐ internal

**IP address type** (i)

**Listeners**

A listener is a process that checks for connection requests, using the protocol and port that you configured.

| Load Balancer Protocol            | Load Balancer Port              |
|-----------------------------------|---------------------------------|
| <input type="text" value="HTTP"/> | <input type="text" value="80"/> |

[Add listener](#)

**Availability Zones**

Specify the Availability Zones to enable for your load balancer. The load balancer routes traffic to the targets in these Availability Zones only. You can specify only one subnet per Availability Zone. You must specify subnets from at least two Availability Zones to increase the availability of your load balancer.

**VPC** (i)

**Availability Zones**

- ☒ **eu-west-2a**   
IPv4 address (i) Assigned by AWS
- ☒ **eu-west-2b**   
IPv4 address (i) Assigned by AWS
- ☐ **eu-west-2c**

Figure A.18: ELB configuration (screenshot from 30.3.2020, source: author)

**Step 2: Configure Security Settings**

**Select default certificate**

AWS Certificate Manager (ACM) is the preferred tool to provision and store server certificates. If you previously stored a server certificate using IAM, you can deploy it to your load balancer. [Learn more](#) about HTTPS listeners and certificate management.

**Certificate type** (i) ☒ Choose a certificate from ACM (recommended)  
☐ Upload a certificate to ACM (recommended)  
☐ Choose a certificate from IAM  
☐ Upload a certificate to IAM

[Request a new certificate from ACM](#)  
AWS Certificate Manager makes it easy to provision, manage, deploy, and renew SSL Certificates on the AWS platform. ACM manages certificate renewals for you. [Learn more](#)

**Certificate name** (i)

**Select Security Policy**

**Security policy** (i)

Figure A.19: ELB HTTPS certificate security settings (screenshot from 30.3.2020, source: author)

**Step 4: Configure Routing**

Your load balancer routes requests to the targets in this target group using the protocol and port that you specify, and performs health checks on the targets using these health check settings. Note that each target group can be associated with only one load balancer.

**Target group**

**Target group** (i)

**Name** (i)

**Target type** (i) ☒ Instance  
☐ IP  
☐ Lambda function

**Protocol** (i)

**Port** (i)

**Health checks**

**Protocol** (i)

**Path** (i)

**Advanced health check settings**

**Port** (i) ☒ traffic port  
☐ override

**Healthy threshold** (i)

**Unhealthy threshold** (i)

**Timeout** (i)  seconds

**Interval** (i)  seconds

**Success codes** (i)

Figure A.20: ELB routing (screenshot from 30.3.2020, source: author)

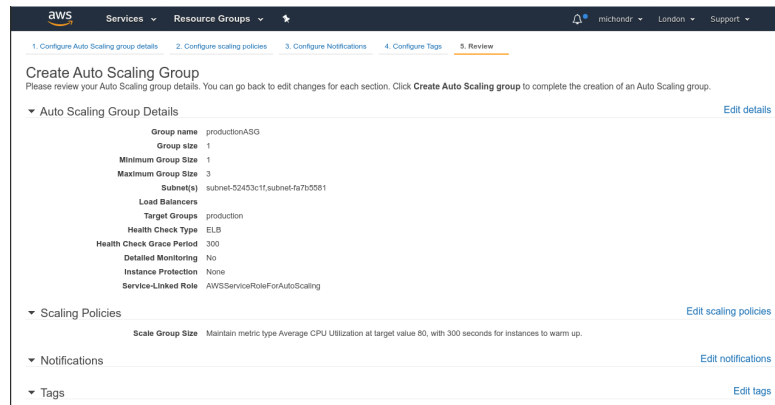


Figure A.21: Auto Scaling Group - review (screenshot from 30.3.2020, source: author)

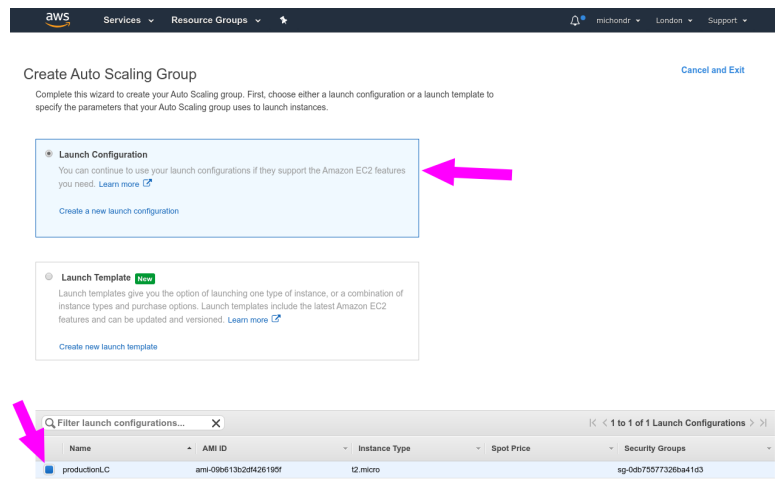


Figure A.22: Auto Scaling Group - create new launch configuration (screenshot from 30.3.2020, source: author)



# B. Navigation of VM management in GCP

In this annex more detailed screenshots are added, visualizing the user experience of navigating in the GCP Cloud Engine instance creation process.

Google Cloud Platform michondr

Search resources and products

Create an instance group

To create an instance group, select one of the options:

- New managed instance group**  
Create a group of identical VM instances from an existing template. Manage VM instances as a single entity.
- New unmanaged instance group**  
Create a group of unique VM instances without using a template. Add and remove VM instances manually.

Organise VM instances in a group to manage them together. [Instance groups](#)

**Name** ⓘ  
Name is permanent  
instance-group-1

**Description** (Optional)  
[Text area]

**Location**  
To ensure higher availability, select a multiple zone location for an instance group. [Learn more](#)

☒ Single zone  
☐ Multiple zones

**Region** ⓘ  
Region is permanent  
europe-west3 (Frankfurt)

**Zone** ⓘ  
Zone is permanent  
europe-west3-c

**Specify port name mapping** (Optional)

**Instance template** ⓘ  
f1-micro-preemptible-ubuntu-http-allowed

**Number of instances**  
Based on auto-scaling configuration

**Auto-scaling**  
Use autoscaling to allow automatic resizing of this instance group for periods of high and low load. [Autoscaling groups of instances](#)

**Auto-scaling mode**  
Auto-scale

**Autoscaling metrics**  
Use metrics to determine when to autoscale the group. [Autoscaling policy and target utilization](#)

CPU utilisation: 60% (default)

+ Add new metric

**Cool-down period** ⓘ  
Specify how long to wait for a new instance before taking its metrics into account. [Cool-down period](#)

60 seconds

**Minimum number of instances** ⓘ  
1

**Maximum number of instances** ⓘ  
5

Delete auto-scaling configuration

[Less](#)

**Autohealing** ⓘ  
**Health check**  
No health check

Compute Engine will recreate VM instances only when they're not running.

[Advanced creation options](#)

same configuration as new VM creation

Figure B.1: GCP new instance group creation (screenshot from 6.4.2020, source: author)

In the backend creation the Instance Group is selected, instance-group-1 in this case, based on the f1-micro-preemptible-ubuntu-http-allowed instance template.

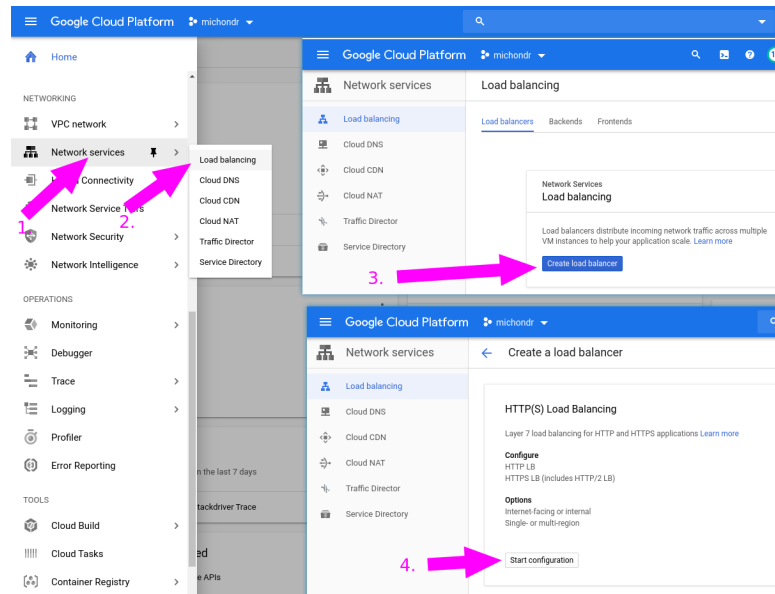


Figure B.2: GCP load balancer creation (screenshot from 6.4.2020, source: author)

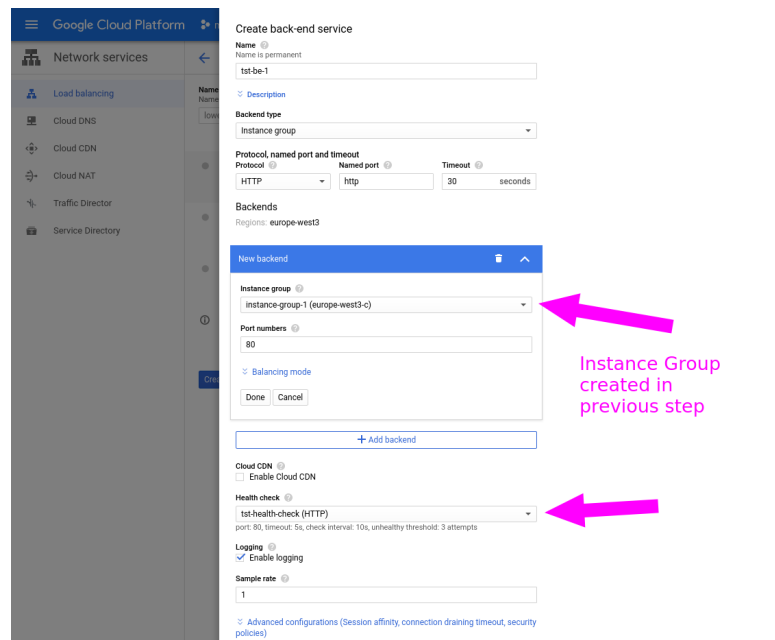


Figure B.3: GCP load balancer backend creation (screenshot from 6.4.2020, source: author)



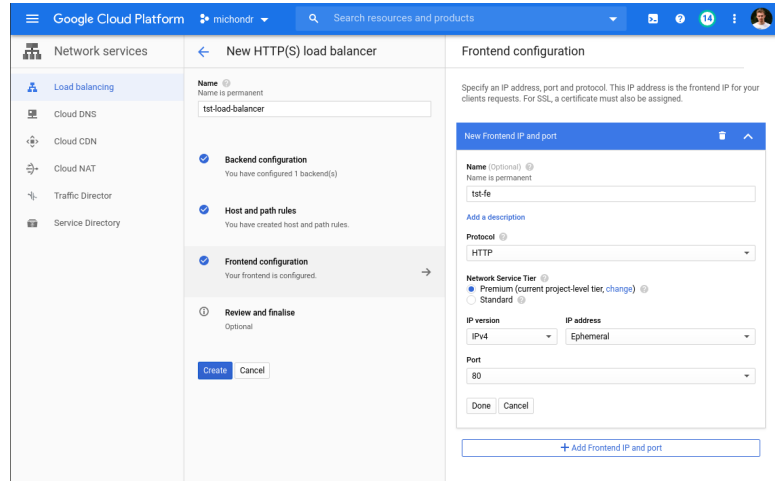


Figure B.4: GCP load balancer frontend creation (screenshot from 6.4.2020, source: author)

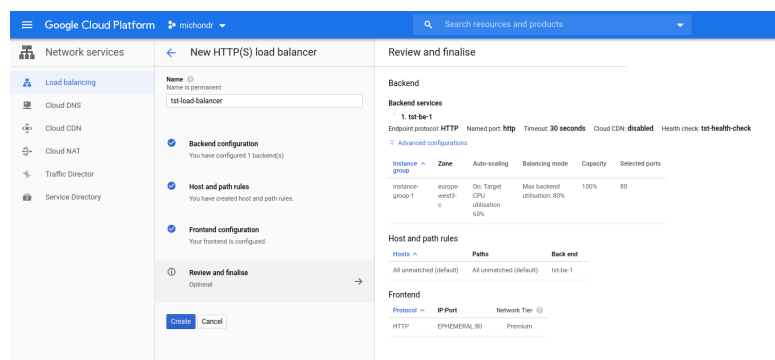


Figure B.5: GCP load balancer summarization (screenshot from 30.4.2020, source: author)  
(screenshot from 6.4.2020, source: author)



# C. Faulty access to Roles of GCP Service Account

This screenshot shows how to access roles of Service Account when there is no option to edit or list the Roles in the GCP web interface. The only possible solution is to use the CLI tool gcloud.

Here is example of using the tool to list the roles:

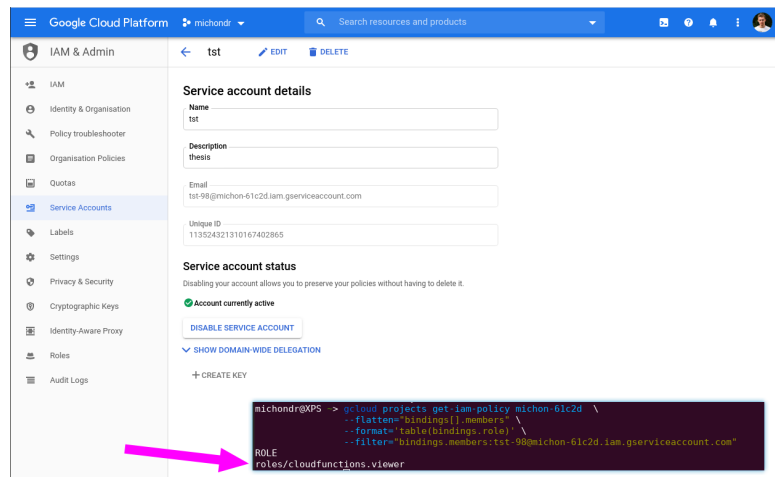


Figure C.1: GCP missing roles in Service Account (screenshot from 26.4.2020, source: author)